VOID HACKS() 8.0 · TRACK: ABHEDYA — CYBER SECURITY & DIGITAL FORENSICS

Frontend Architecture & UI Specification

दर्शन परत

The implementation contract for the React investigation console: eight routes, page-by-page layouts and states, Cytoscape rules, the component-to-API contract, error and empty-state copy, accessibility and visual language, the blind-victim test mode, and the build order.

VERSION	1.0 — implementation contract
STATUS	Frozen for v1.0
OWNER	M2 — Frontend
COMPANION TO	Backend API Contract v1.0 · PRD v2.1
ASSOCIATED ORG	Indore Police Commissionerate
EVENT	Void Hacks() 8.0
DATE	02 October 2026

[PS] problem statement · [DEC] team decision · [API] fixed by the API contract · [A11Y] accessibility

Version 1.0 · Implementation contract

This document is the implementation contract for the **React frontend** of ABHEDYA-CHAKRA. It is the third of three contracts:

Document	Owns
Product Requirements Document v2.1	What the product must do and why — algorithms, thresholds, dataset reality
Backend Architecture & API Contract v1.0	The 45 routes, the response envelope, the error codes, the EvidenceObject
This document	How the investigator sees it: pages, components, states, interactions, and the exact API call behind every pixel

Where this document and the API contract disagree on a payload, **the API contract wins** and this document is corrected. Where this document and the PRD disagree on meaning — what a layer is, what a signal proves — **the PRD wins**.

0.1 The frontend's standing prohibitions

The frontend consumes the FastAPI backend **exclusively** through the documented REST API. It **MUST NOT**:

#	Prohibition	Why it matters here
1	Access DuckDB directly	There is no browser-reachable database. There is no connection string to leak
2	Access CSV / Parquet / CSR directly	274 MB of CSV has no business in a browser tab
3	Calculate the Mule Risk Index	One scoring implementation, in `detect/scoring.py`. A second one in JavaScript would drift within a day and put a different number on screen than in the PDF
4	Calculate L1 / L2 / L3	Layers are victim-relative and come from a ten-input classifier. A hop counter in the UI is exactly the forbidden "classify by hop depth alone"
5	Perform graph traversal	The browser never holds the adjacency. It renders the bounded subgraph the backend returned
6	Independently decide an account is suspicious	No client-side thresholding, no "if risk > 80 add a skull icon the backend never sanctioned"
7	Invent evidence	Every account number, amount, timestamp and reason string on screen is a backend field, rendered — never composed
8	Duplicate backend detection logic	If a rule exists in `detect/`, it must not exist in `src/`

> **The single test:** if you deleted the backend and the UI could still produce a number, that number is a bug.

0.2 What the frontend is allowed to own

Formatting (₹ grouping, date display, account masking) · layout and density · selection, zoom, pan, playback position · filter and pagination requests · caching of responses keyed by `dataset_fingerprint` · optimistic pin/unpin that reconciles against the server response · copy that explains a backend state in plain language.

Formatting is not calculation. `formatCurrency(250000) → "₹2,50,000"` is the frontend's job. `sum(transactions.map(t => t.amount))` is not.

0.3 Markers

Marker	Meaning
[PS]	Derives from the official problem statement
[DEC]	Team engineering decision
[API]	Behaviour fixed by the Backend API Contract v1.0
[A11Y]	Accessibility requirement
MUST / MUST NOT	Non-negotiable

---

1. Frontend Technology

1.1 Stack

Concern	Choice	Rationale
Framework	**React 18**	Team familiarity; the component model maps cleanly onto the inspector / graph / timeline split
Build	**Vite**	Sub-second HMR matters in a 36-hour build
Language	**TypeScript preferred, JavaScript acceptable** [DEC]	Types generated from the API contract catch a renamed field at compile time instead of during the demo. If the team is faster in JS, use JSDoc typedefs for the API response shapes at minimum
Graph	**Cytoscape.js** + `cytoscape-dagre`	Handles 500–1,500 elements comfortably, mature selection/style API, works offline
HTTP	**Axios**, one shared instance	Interceptors give a single place for envelope unwrapping, the request id and error mapping
Routing	**React Router v6**	Eight routes, nested case routes
Styling	**Tailwind CSS** + a small design-token layer	Density is a first-class requirement; utilities keep table-heavy screens consistent
State	**React Context +** `useReducer`; TanStack Query optional for the server cache	Avoids Redux ceremony in a 36-hour build
Tables	**TanStack Table** (headless) + row virtualization	A single account can have thousands of transactions

No charting library is required. The three visualisations are the graph (Cytoscape), the timeline (hand-rolled SVG) and the risk bars (CSS). Adding a chart dependency for six bars is a bad trade. **[DEC]**

Everything must work offline. Fonts, icons and the Cytoscape bundle are local dependencies, never CDN links. The demo runs with Wi-Fi switched off. **MUST**

1.2 Directory structure

```
frontend/
├── index.html
├── vite.config.js            # dev proxy: /api -> http://127.0.0.1:8000
├── tailwind.config.js
└── src/
    ├── main.jsx              # bootstrap, router, providers
    ├── App.jsx               # AppShell + route table
    │
    ├── components/
    │   ├── layout/           # AppShell, Sidebar, TopBar, StatusBar, SystemStatus
    │   ├── dashboard/        # StatCard, DatasetStatusCard, QuickInvestigate
    │   ├── investigation/    # InvestigationSummary, ProgressTracker, BlindTestPanel
    │   ├── graph/            # InvestigationGraph, GraphToolbar, GraphLegend,
    │   │                     # NodeInspector, EdgeInspector, cytoscapeStyles.js
    │   ├── timeline/         # Timeline, TimelineControls, TimeRangeSelector,
    │   │                     # TransactionEvent
    │   ├── account/          # AccountOverview, AccountStatistics, FlowPanel, RiskCard,
    │   │                     # RiskBreakdown, ClassificationCard, TerminalEvidenceCard,
    │   │                     # TransactionTable
    │   ├── evidence/         # EvidenceBasket, EvidenceItem, EvidenceSummary, PinButton
    │   ├── documents/        # DocumentPanel, DocumentStatus, FreezeAccountPicker
    │   └── common/           # LoadingState, ErrorState, EmptyState, Modal, Toast,
    │                         # Button, Badge, Tooltip, Pagination, ProvenanceChip
    │
    ├── pages/
    │   ├── Dashboard.jsx
    │   ├── Investigation.jsx
    │   ├── CaseList.jsx
    │   ├── CaseDetail.jsx
    │   ├── AccountDetails.jsx
    │   ├── Evidence.jsx
    │   ├── Documents.jsx
    │   └── System.jsx
    │
    ├── services/             # the ONLY place fetch/axios appears
    │   ├── api.js            # axios instance, envelope unwrap, error mapping
    │   ├── health.js         ingest.js      accounts.js
    │   ├── trace.js          graph.js       risk.js
    │   ├── timeline.js       evidence.js    cases.js
    │   └── documents.js
    │
    ├── hooks/
    │   ├── useInvestigation.js   # orchestration: case -> trace -> graph
    │   ├── useGraph.js           # Cytoscape lifecycle + incremental updates
    │   ├── useTimeline.js        # playback clock, scrub position, event windowing
    │   ├── useCase.js            # case rehydration and mutation
    │   ├── useAccount.js         # account detail + risk + classification fan-out
    │   ├── useHealth.js          # bootstrap probe + poll cadence
    │   └── useDebounce.js
    │
    ├── context/
    │   ├── InvestigationContext.jsx
    │   ├── SystemContext.jsx     # health, profile, fingerprint
    │   └── ToastContext.jsx
    │
    ├── utils/
    │   ├── formatCurrency.js     # 250000 -> "₹2,50,000"  (Indian grouping)
    │   ├── formatDate.js         # ISO -> "22 Sep 2026, 11:02:41"
    │   ├── formatAccount.js      # masking, grouping, clipboard text
    │   ├── formatDuration.js     # 412 -> "6m 52s"
    │   ├── errorCopy.js          # error.code -> user-facing sentence (section 30)
    │   └── layerTheme.js         # layer/role/band -> token names (lookup, not logic)
    │
    ├── types/api.d.ts            # generated or hand-written from the API contract
    ├── styles/globals.css        # tokens + Tailwind layers
    └── assets/                   # local fonts, SVG icons, emblem
```

Module rules [DEC]

1. `services/*` is the **only** place `axios` / `fetch` appears. `grep -r "fetch(\|axios" src/components src/pages` must return nothing.
2. `components/*` receive data as props or from a hook; they never call services directly.
3. `utils/*` contains **pure formatting functions only** — no thresholds, no classification, no arithmetic over API data.
4. `layerTheme.js` is a lookup table (`L1 -> {label, token, icon}`), not a decision function.

5. No component imports from a page; no page imports from another page.

1.3 Vite dev proxy

```
// vite.config.js
export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    proxy: { "/api": { target: "http://127.0.0.1:8000", changeOrigin: true } },
  },
});
```

All frontend code calls **relative** `/api/...` URLs. No component or service hard-codes `http://localhost:8000`, so the same build works behind the dev proxy, behind a reverse proxy, or served as static files by the backend itself. **MUST**

---

2. Design Goal

ABHEDYA-CHAKRA is a **cyber-financial investigation console** — not a dashboard product, and not a game.

It must feel	It must not feel
Professional — it produces documents that freeze money	Neon / cyberpunk
Forensic — every figure is attributable	Animated for its own sake
Investigative — built around a question, not around charts	Gamified (badges, trophies, "threat level" dials)
Information-dense — an investigator reads, does not scroll	Decorative — whitespace that costs a row of evidence
Fast — the blind test is timed	Skeleton-heavy — shimmering boxes everywhere
Readable — at 1280×720, under projector light	Low-contrast "aesthetic" greys
Trustworthy — it never overclaims	Alarmist — flashing red, sirens, "CRIMINAL DETECTED"

The interface exists to answer one question as fast as possible:

> **"Where did the victim's money go?"**

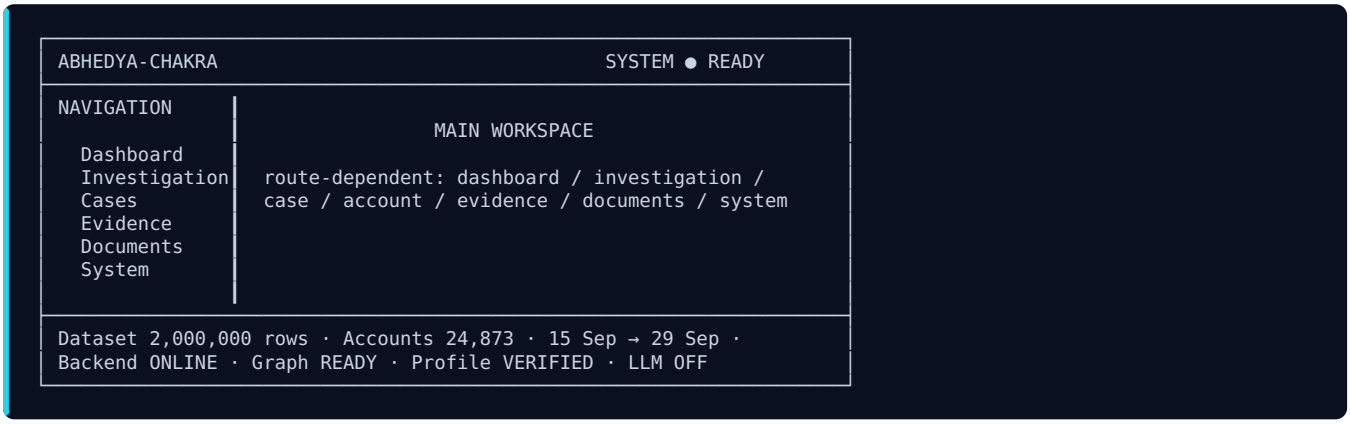
Every screen is judged against that sentence. If an element does not help answer it, or help prove the answer, it is removed.

2.1 Design rules that follow from the goal

#	Rule
1	**The graph gets the most pixels.** On the investigation page it is the largest region at every breakpoint
2	**Numbers are right-aligned, tabular-figure, and grouped.** Amounts, counts, IDs and scores are compared down a column
3	**Nothing animates longer than 200 ms**, and nothing animates on data arrival except opacity. Layout must never shift under a reading eye
4	**One accent colour, used for interaction only.** Colour elsewhere means data: severity, layer, state
5	**Every number has a provenance affordance** — hover or click reveals where it came from (section 35)
6	**No modal on the critical path.** The blind test must be completable without dismissing anything
7	**Loading states say what is happening**, in the backend's own stage names (section 8)
8	**Empty is not failure.** An empty trace is a finding; the UI says so and offers the next action (section 31)

3. Main Application Layout

3.1 The shell



3.2 Regions

Region	Component	Behaviour
Top bar	`TopBar`	Product name + Devanagari mark, global search, `SystemStatus` pill, demo-mode toggle
Navigation	`Sidebar`	Seven destinations (section 38). Collapses to a 56 px icon rail below 1440 px, collapsible by the user at any width; state persists in `localStorage`
Workspace	route outlet	Owns all remaining space and **scrolls internally**, so the status bar and search are always reachable
Status bar	`StatusBar`	Dataset facts and connection state, all from `/api/health` and `/api/ingest/profile`. Never hard-coded

3.3 `SystemStatus` pill — the one indicator always on screen

Backend `data.status`	Pill	Token	Secondary text
`healthy`	**READY**	`--ok`	"24,873 accounts · 2,000,000 rows"
`degraded`	**READY (caveats)**	`--warn`	e.g. "profile not verified" / "LLM offline"
`loading`	**INDEXING 72%**	`--info`	Current stage name from the ingest job
`unavailable`	**NO DATASET**	`--muted`	"Load a dataset to begin"
network failure	**BACKEND OFFLINE**	`--critical`	"Retrying in 5 s"

The pill is a **button** that navigates to `/system`. [A11Y] It carries `role="status"` and `aria-live="polite"`, and its state is conveyed by text, not colour alone.

3.4 Shell behaviour rules

1. The shell renders **before** `/api/health` resolves, with the pill in a neutral "connecting" state. A blank page while a probe is in flight is a demo risk. **MUST**
2. If health returns `unavailable`, every route except `/system` and `/` renders the "engine not ready" `EmptyState` with a link to the system page — no spinner purgatory.

3. The shell watches `meta.dataset_fingerprint`. If it changes mid-session, a non-blocking toast appears — "Dataset changed. Cases from the previous dataset are read-only." — and cached responses are dropped. **[API]**
4. Keyboard: `/` focuses global search, `g i` investigation, `g c` cases, `?` the shortcut sheet. [A11Y]

---

4. Dashboard Page

Route `/` · `pages/Dashboard.jsx`

Purpose: prove the system is real, and let the investigator start in a single action.

4.1 Hero — investigate immediately



- Autofocus on mount. **Enter submits.** The judge should never need the mouse to start.
- Inline validation on blur against `^[0-9]{12}$`; once 12 digits are present, a debounced (300 ms) `GET /api/accounts/{no}/exists` catches a typo before the trace runs.
- Submitting navigates to `/investigation?account=…&autostart=1`.

4.2 System statistics strip

Tile	Example	Source
Transactions	`2,000,000`	`health.transactions`
Accounts	`24,873`	`health.accounts`
Dataset	`15 days · 15 Sep → 29 Sep 2026`	`profile.date_range`
Backend	`ONLINE`	HTTP success on `/api/health`
Graph	`READY`	`health.graph_loaded`
Profile	`VERIFIED` / `MISMATCH`	`profile.verification.profile_check_passed`

Every value comes from `GET /api/health` and `GET /api/ingest/profile`. Hard-coding any of them is a defect — the number on screen must change when the dataset changes. **MUST**

A tile with no value yet shows `—`, never `0`, and a dash is never styled like a value.

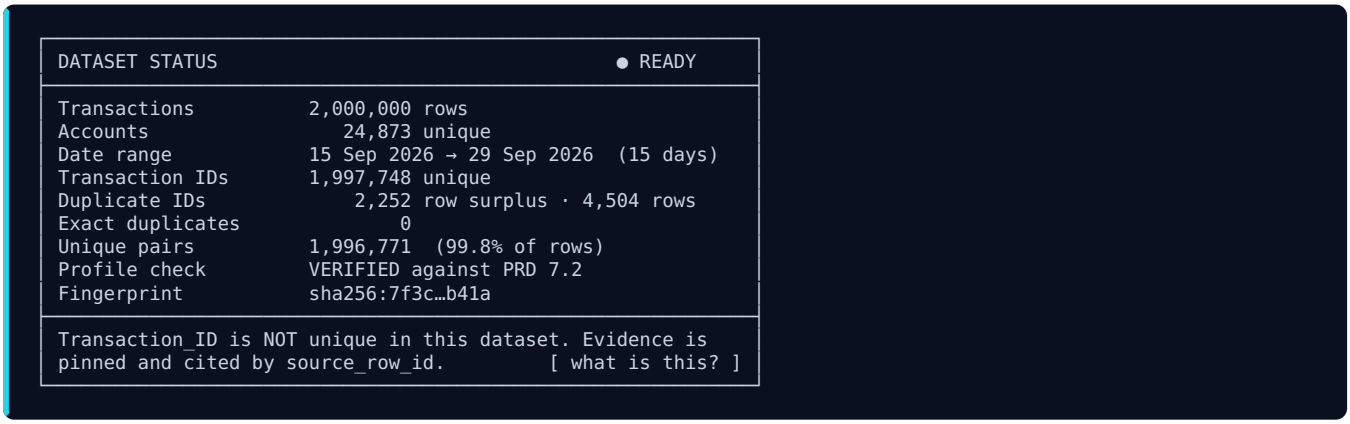
4.3 The rest of the dashboard

Block	Content	Source
`DatasetStatusCard`	Section 5	`/api/ingest/profile`
Risk distribution	20-bucket MRI histogram + band counts, clickable into a filtered search	`/api/risk/distribution`
Recent cases	Last 5: id, victim, status, evidence count	`/api/cases`
Top-risk accounts	10 highest MRI with layer hint and role	`/api/risk/distribution.top`
Performance	Measured latencies, or **"not yet measured"**	`/api/benchmarks`

The performance block renders `—` for every `null`. It must never display a placeholder number, an estimate, or a figure measured on another machine. **[API] [PS]**

5. Dataset Status Card

`components/dashboard/DatasetStatusCard.jsx` — also embedded in `/system`.



5.1 Field map

Label	Path in `/api/ingest/profile`	Notes
Transactions	`row_count`	
Accounts	`unique_accounts`	
Date range	`date_range.start` / `.end` / `.span_days`	
Transaction IDs	`unique_transaction_ids`	
Duplicate IDs	`duplicate_transaction_ids.row_surplus` **and** `.rows_affected`	**Show both.** They are different numbers
Exact duplicates	`exact_duplicate_rows`	
Unique pairs	`graph_shape.unique_pairs`	
Profile check	`verification.profile_check_passed`	
Fingerprint	`dataset_fingerprint`	Truncated on screen, full on hover

5.2 Rules

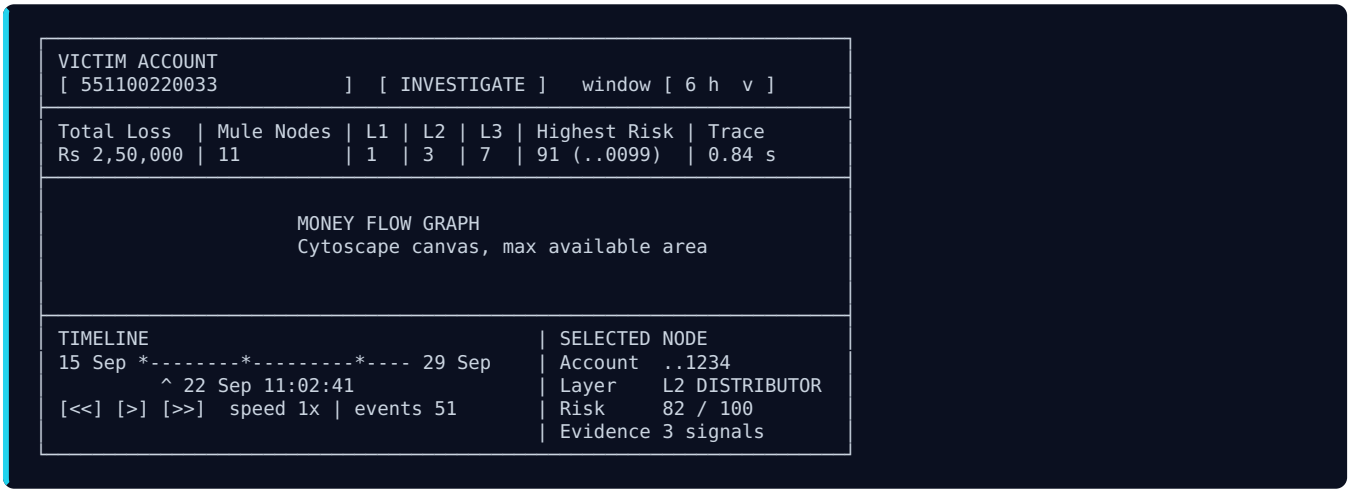
1. **The duplicate-ID line is mandatory and may not be hidden behind a toggle.** It is the fact that shapes the entire evidence model, and a judge who sees it understands immediately that the team read its data. **MUST**
2. `[ what is this? ]` opens a two-sentence popover: "2,252 transaction IDs repeat, affecting 4,504 rows. No row is deleted — every record is pinned and cited by its CSV row number (*`source_row_id`*)."
3. If `verification.profile_check_passed` is `false`, the card becomes a **warning state** listing each mismatch (`key · expected · actual`), and the same banner appears on `/system`. It is never silently swallowed. **[API]**
4. Memory and processing figures appear **only if** `/api/health.memory_mb` and `/api/benchmarks` provide them; otherwise the row is omitted — never rendered as `0`.

6. Investigation Page

Route `/investigation` · `pages/Investigation.jsx` — the most important page in the product.

The blind-victim test carries **40% of the judging weight** [PS]. This page must be fast, reliable, and comprehensible to someone who has never seen it before.

6.1 Layout



6.2 Region specification

#	Region	Size	Component	Notes
1	Search + controls	64 px	`AccountSearch` + window preset	Always visible; never scrolls away
2	Summary strip	88 px	`InvestigationSummary`	Section 9. Populates the instant the trace resolves
3	Graph canvas	**flex: 1, min 420 px**	`InvestigationGraph`	Takes all remaining vertical space
4	Timeline	180 px, collapsible	`Timeline`	Section 16
5	Inspector	380 px right column	`NodeInspector` / `EdgeInspector`	Docks right ≥ 1440 px; stacks below the graph under that

6.3 Page states

State	Trigger	What renders
`idle`	No account entered	Centred `EmptyState`: "Enter a victim account to begin investigation." plus the demo IDs when demo mode is on
`validating`	12 digits entered	Inline spinner on the field only; the page does not change
`investigating`	Trace in flight	`ProgressTracker` (section 8) occupies the graph area; summary tiles show skeletons
`ready`	Trace resolved, nodes > 0	Full layout

State	Trigger	What renders
`empty-result`	Trace resolved, nodes = 0	Section 31: "No outbound flow in this window" + **[Trace upstream instead]** + **[Widen window]**
`truncated`	`summary.truncated === true`	Full layout **plus** a persistent amber bar naming `truncation_reason` and offering the matching remedy
`error`	Any error code	`ErrorState` with the section-30 copy and a retry affordance

6.4 The URL is the state [DEC]

```
/investigation?account=551100220033&case=CASE-2026-001
              &trace=trc_01JA8Z9V3K&node=402938475610&t=1758535361
```

Reloading restores the investigation from `GET /api/trace/{trace_id}` without re-running the traversal; sharing the URL shows a teammate the same screen. During judging this is the difference between "let me redo that" and pressing F5.

---

7. Investigation Workflow

7.1 The ten steps

Step	Action	API	Failure handling
1	**Validate account ID**	`GET /api/accounts/{no}/exists`	Invalid format → inline field error, no network call. Unknown account → `ACCOUNT_NOT_FOUND` with backend `suggestions` rendered as "did you mean" chips
2	**Create or retrieve case**	`POST /api/cases`	If the URL already carries `case`, `GET /api/cases/{id}` instead and skip to step 3
3	**Trace the victim**	`POST /api/trace` with `case_id`	`TRACE_LIMIT_EXCEEDED` → clamp and explain; empty result → `empty-result` state with the upstream fallback
4	**Load the graph**	`GET /api/graph/{account}?case_id=…`	Bounded by the backend; render `stats.truncated` honestly
5	**Load the summary**	Already in the trace response (`summary`, `performance`)	Never a second request — the numbers are in the trace payload
6	**Render nodes and edges**	—	Cytoscape batch add, layout once (section 29)
7	**Display the timeline**	`GET /api/timeline?case_id=…`	Fetched in parallel with step 4, not serially
8	**Allow node selection**	—	Selection is local state; no refetch of the graph
9	**Load account detail on select**	`GET /api/accounts/{no}` + `/risk/{no}` + case-relative `/classification`	Fan-out in parallel, 300 ms debounce against rapid clicking, LRU-cached by `(fingerprint, account)`
10	**Pin evidence**	`POST /api/evidence/basket`	Optimistic add, reconciled against `items_added` / `skipped`

7.2 Orchestration (the one place this logic lives)

```
// hooks/useInvestigation.js — simplified
async function investigate(accountNo, { windowSec, caseId }) {
  dispatch({ type: "STAGE", stage: "validating" });
  const { exists } = await accounts.exists(accountNo);          // step 1
  if (!exists) throw new ApiError("ACCOUNT_NOT_FOUND", { accountNo });

  dispatch({ type: "STAGE", stage: "case" });
  const kase = caseId ? await cases.get(caseId)                 // step 2
                      : await cases.create({ victim_account: accountNo,
                                             incident_time: incidentTime });

  dispatch({ type: "STAGE", stage: "tracing" });
  const trace = await traceApi.run({ victim_account: accountNo, // step 3
                                     case_id: kase.case_id,
                                     window_sec: windowSec });

  dispatch({ type: "STAGE", stage: "graph" });
  const [graph, tl] = await Promise.all([                       // steps 4 + 7
    graphApi.get(accountNo, { case_id: kase.case_id }),
    timelineApi.get({ case_id: kase.case_id }),
  ]);

  dispatch({ type: "READY", kase, trace, graph, timeline: tl,
             timings: { trace: trace.performance.total_ms,
                        graph: graph.__meta.processing_ms } });
}
```

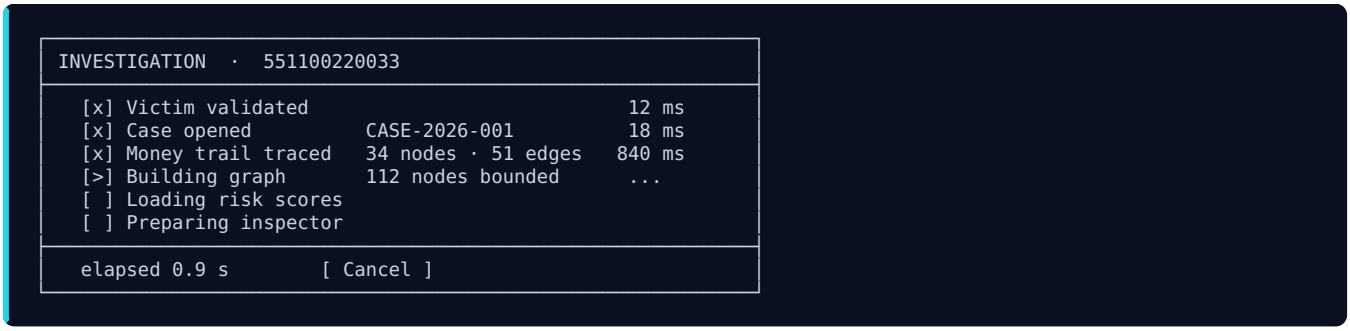
Rules

1. **This sequence exists in exactly one module.** No page or component re-implements it. **MUST**
2. Steps 4 and 7 run **in parallel**; serialising them adds ~400 ms to the demo for no reason.
3. Every stage transition is dispatched before the await, so the progress tracker is truthful rather than retroactive.
4. `autostart=1` in the URL runs the whole sequence on mount — that is the blind-test path (section 36).
5. On any error the reducer stores `{code, message, details, requestId}` and the page renders `ErrorState`. The partial results already received stay visible when they are still valid (a failed timeline does not blank the graph).

---

8. Investigation Loading State

"Loading…" is forbidden on this page. [DEC] A judge watching a blank spinner for 900 ms learns nothing; a judge watching named stages learns that the system has an architecture.



8.1 Stage mapping — real stages only

UI line	Becomes ✓ when	Timing shown
Victim validated	`/exists` resolves	its `meta.processing_ms`
Case opened	`POST /api/cases` returns 201	its `meta.processing_ms`
Money trail traced	`POST /api/trace` resolves	`performance.total_ms`
Building graph	`GET /api/graph` resolves	its `meta.processing_ms`
Loading risk scores	`POST /api/risk/batch` resolves	its `meta.processing_ms`
Preparing inspector	First node auto-selected and detail loaded	its `meta.processing_ms`

Rules

1. **Do not fake progress.** No timer-driven bar, no interpolation, no "almost there". A step is pending, active, or complete, and the elapsed clock is real. **MUST**
2. Each completed line shows the **backend-reported** duration (`meta.processing_ms`), not a stopwatch around the fetch. Those numbers are the ones quoted in section 41.
3. During ingestion (`/api/ingest/status`), the same component renders the backend's own nine stage names and `progress` integer — again, never interpolated. **[API]**
4. A step that takes longer than 3 s gains a secondary line explaining why ("large neighbourhood — applying bounds"), sourced from `meta.warnings` when present.
5. `[ Cancel ]` aborts the in-flight requests via `AbortController` and returns to `idle`. A hung demo must always be escapable.

9. Investigation Summary

```
VICTIM 551100220033   ·   CASE-2026-001   ·   window 6 h

Total Loss | Mule Nodes | L1 | L2 | L3 | Highest Risk | Trace
Rs 2,50,000  | 11          |  1 |  3 |  7 | 91  ..0099   | 0.84 s

traced Rs 2,50,000 · terminal Rs 94,000 · held (est.) Rs 38,400
23 accounts unclassified · not truncated
```

9.1 Field map — all values are backend fields, rendered

Tile	Source	Format
Victim Account	`trace.victim`	Masked per the display setting, full on hover
Total Traced Amount	`trace.total_traced_value`	`formatCurrency`
L1 Collector Count	`trace.summary.l1_count`	integer
L2 Distributor Count	`trace.summary.l2_count`	integer
L3 Terminal Count	`trace.summary.l3_count`	integer
Mule Nodes	`l1 + l2 + l3` (a display sum of backend counts, not a reclassification)	integer
Highest Risk Account	the trace node with max `risk_score` — selected, not computed	`91 · ..0099`, clicking selects it in the graph
Trace Latency	`trace.performance.total_ms`	`0.84 s`
Secondary line	`terminal_value`, `held_value_total`, `unclassified_count`, `truncated`	—

9.2 Rules

1. **The frontend displays the backend result; it does not calculate these values.** The only arithmetic permitted is summing three backend-provided layer counts for a "mule nodes" convenience figure, and picking the max of a returned list. **MUST**
2. `unclassified_count` is shown, not hidden. 23 accounts in the trace that the classifier would not label is an honest and informative number; concealing it would imply a precision we do not claim.
3. If `summary.truncated` is true, the strip shows a **TRUNCATED** badge linking to the explanation bar. Never present a truncated total as a complete one. **[API]**
4. Trace latency is displayed **always**, not only when it is fast. This is the number the performance criterion is judged on.
5. Tiles are `<button>`s where they navigate (highest-risk, layer counts filter the graph). [A11Y]

10. Graph Visualization

`components/graph/InvestigationGraph.jsx` — the centrepiece.

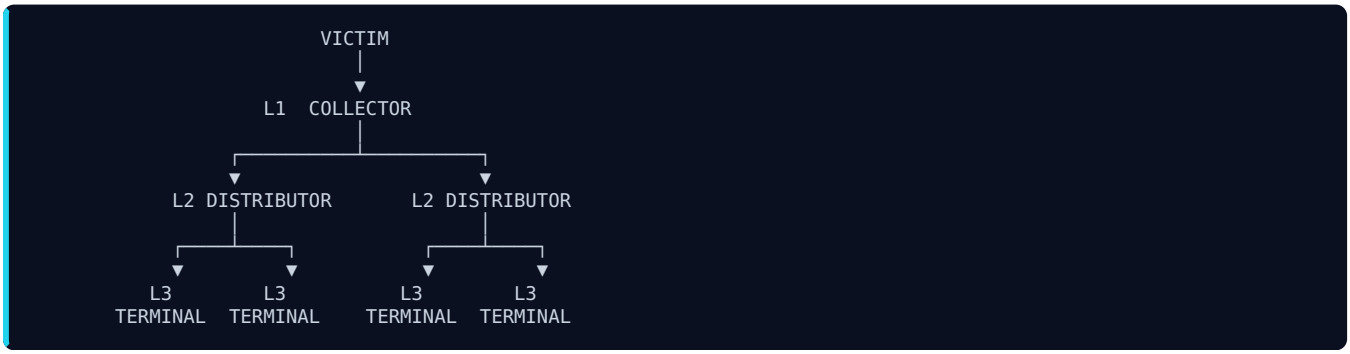
10.1 Node types and visual encoding

Type	Shape	Fill	Border	Icon	Label
VICTIM	round-rectangle, largest	`--victim` (blue)	3 px solid	⌖ crosshair	`VICTIM` + masked account
L1 COLLECTOR	ellipse	`--l1` (deep amber)	2 px	↓ into-circle	`L1` + account
L2 DISTRIBUTOR	ellipse	`--l2` (orange)	2 px	⤲ split	`L2` + account
L3 TERMINAL	diamond	`--l3` (red)	2 px	⇥ endpoint	`L3` + account
NORMAL / UNKNOWN	small ellipse	`--neutral` (grey)	1 px	none	account only
HUB / AGGREGATOR	hexagon	`--neutral`	2 px dashed	⬡	`HUB` + degree

Encoding rules

1. **Node size ∝ traced value** (not risk, not degree), clamped to a 24–64 px range. Money is the subject of the investigation, so money is the size channel.
2. **Border thickness ∝ risk band**, and the band is **also** printed as text in the label on hover. Colour is never the only carrier. [A11Y]
3. **Terminal evidence** adds a small corner marker, with a tooltip that says "carries terminal indicators — evidence, not proof". [PS]
4. **Selected** state: 3 px accent ring + the inspector opens. **Hovered**: label expands to full account number.
5. Edges: width ∝ `traced_amount` (clamped), `is_traced: true` edges are solid and accented, context edges are thin and grey; arrowheads always show direction of money.

10.2 Conceptual structure the layout must preserve



Layer must read **top-to-bottom** with `dagre` (`rankDir: TB`), so hop depth is visually obvious without reading a single label. Nodes are ranked by `hop`, not by role — an account that is L2 at hop 2 sits at rank 2.

10.3 Legend

Always visible, bottom-left, collapsible: the six node types with shape, colour **and** name; edge styles (traced vs context); the size channel ("size = traced value"); and a one-line caveat: "Layers are relative to this victim."

---

11. Graph Rules

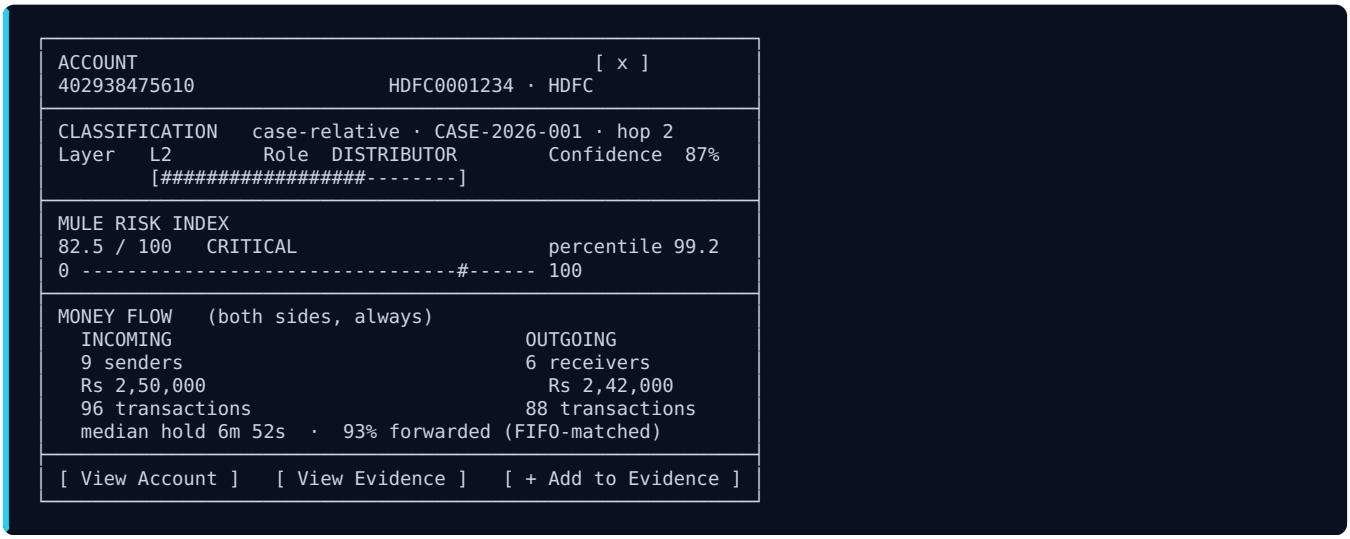
#	Rule
1	**Never render the entire 2 M-row dataset.** The browser receives a bounded subgraph and nothing else. **MUST**
2	Defaults: `max_nodes = 800`, `max_edges = 1500`, `radius = 2` — requested from the backend, enforced by the backend. **[API]**
3	The graph must stay usable at **500+ nodes / 1,500+ edges**: pan and zoom at ≥ 30 fps, selection in < 100 ms
4	When `stats.truncated` is true, show the amber bar with `truncation_reason` and the matching remedy button (narrow window · raise min value · reduce radius)
5	Nodes with `hidden_neighbors > 0` get an expand affordance (double-click or the `+` badge) that calls `/api/graph/{no}/expand` and **patches** the graph — it never reloads it
6	Hub nodes are rendered but never auto-expanded, and are excluded from the freeze picker by default **[API]**
7	Labels auto-hide below zoom 0.6 and can be toggled manually; the toolbar state persists per session

11.1 Required interactions

Control	Keyboard	Behaviour
Zoom in / out	`+` / `-`, wheel	Cursor-anchored, 0.2×–4×
Pan	drag, arrow keys	
Fit to screen	`f`	Animated 150 ms
Reset view	`r`	Fit + clear selection + restore default filters
Focus selected	`z`	Centres and zooms to the selection and its immediate neighbours
Select node / edge	click, `Tab` + `Enter`	Opens the matching inspector [A11Y]
Hide / show labels	`l`	
Expand node	double-click, `e`	`/api/graph/{no}/expand`
Isolate subgraph	`i`	Hides everything except the selection's connected component **within the current response** — a view filter, never a new traversal
Export view	`⌘/Ctrl+S`	PNG of the canvas + the backing JSON (useful in the case file)

"Isolate" is a visibility filter over already-returned elements. It must not compute reachability over anything the backend did not send. **MUST**

12. Graph Node Information — the Node Inspector



12.1 Data sources

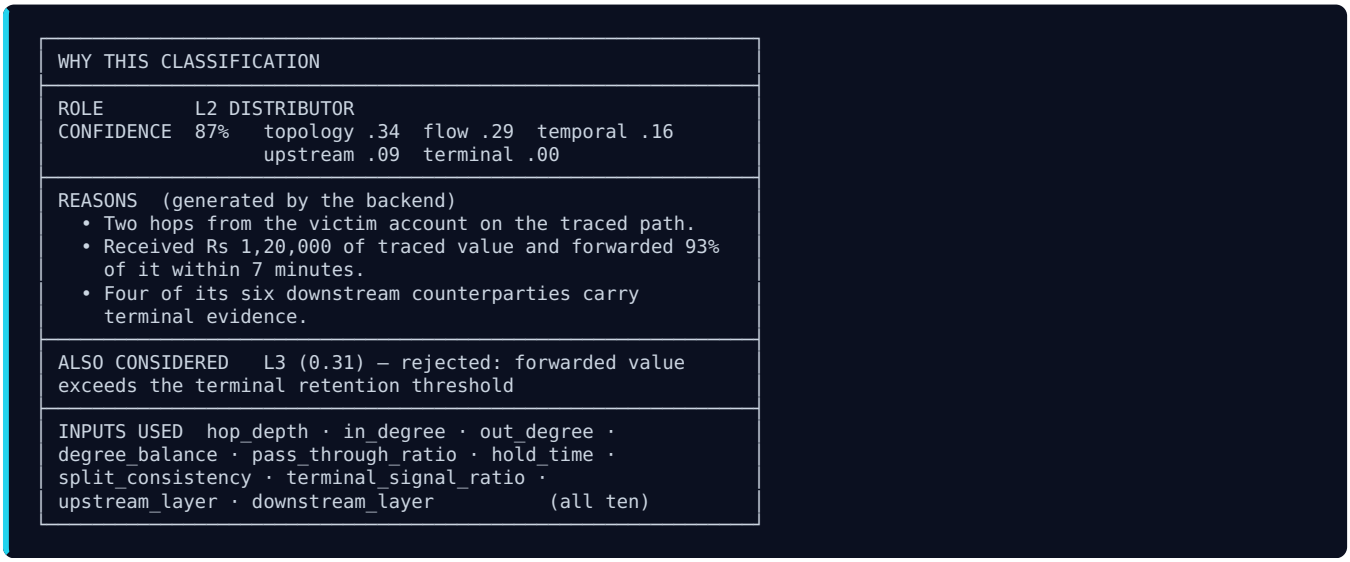
Block	Endpoint	Notes
Header, IFSC, bank	`GET /api/accounts/{no}` → `account`	
Classification	`GET /api/cases/{case_id}/accounts/{no}/classification`	**Case-relative whenever a case exists**
Risk	`GET /api/risk/{no}`	Full component breakdown on expand
Money flow	`GET /api/accounts/{no}` → `statistics` + `flow`	Both sides, always (section 14)
Terminal evidence	`GET /api/accounts/{no}/terminal-evidence`	Lazy: only when the section is expanded

12.2 Rules

1. The inspector opens with data already in the trace payload (layer, role, risk, traced value) and **progressively enriches** as the detail calls resolve. It never shows an empty panel while waiting. **[DEC]**
2. **Inside a case, the inspector shows the case-relative classification only.** The global hint may appear in a secondary "outside this case" row, explicitly labelled. Mixing the two is the one conceptual error that would mislead an investigator. **[API]**
3. Actions: `View Account` → `/accounts/{no}` · `View Evidence` → terminal-evidence section · `Add to Evidence` → `POST /api/evidence/basket` with the account **and** the trace rows that justify it.
4. Closing the inspector clears `?node=` from the URL; selection and URL stay in sync in both directions.
5. The panel is a `<aside role="complementary">` with a focus trap when stacked on narrow screens. [A11Y]

13. Node Role Explanation

The UI must always be able to answer **"why did this account get this label?"** — in the backend's words.



Rules

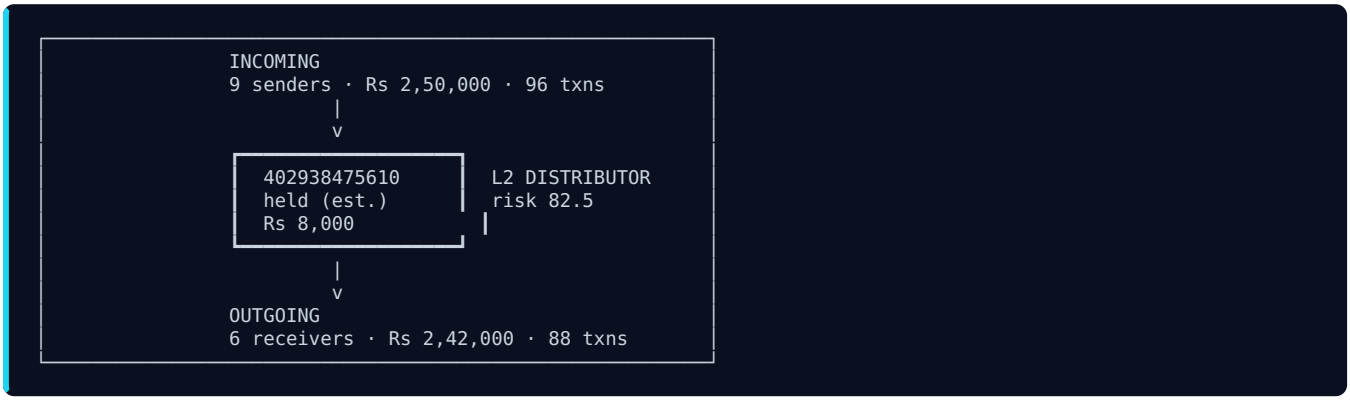
1. **Reason strings are rendered verbatim from `classification.reasons` and `risk.reasons`. The frontend never writes, rewrites, summarises, truncates or re-orders them.** They are the same sentences that appear in the case diary; if the screen and the PDF disagree, the evidence is worthless. **MUST**
2. `confidence_breakdown` is rendered as a stacked bar; the five contributions come from the backend and sum to the confidence. The UI does not normalise them.
3. `alternative_considered` is displayed whenever present. "We considered L3 at 0.31 and rejected it because..." is what separates an opinion from an assertion.
4. `inputs_used` is shown as a chip list — visible proof that ten inputs were consulted, not one. **[PS]**
5. If `confidence < 0.5`, the card carries a neutral "low confidence" note and the node is rendered with a dashed border in the graph. Uncertainty is displayed, not hidden.

---

14. Collector / Distributor UX

> **A collector does not only receive. A distributor does not only send.** In this dataset the average account has ≈80 incoming and ≈80 outgoing transactions — almost every account does both. "Collector" and "distributor" describe a role in this case, derived from position, dominant topology and temporal behaviour — never from a capability. **[PS, PRD §2.4]**

Every suspicious node is therefore rendered with **both sides, always**:

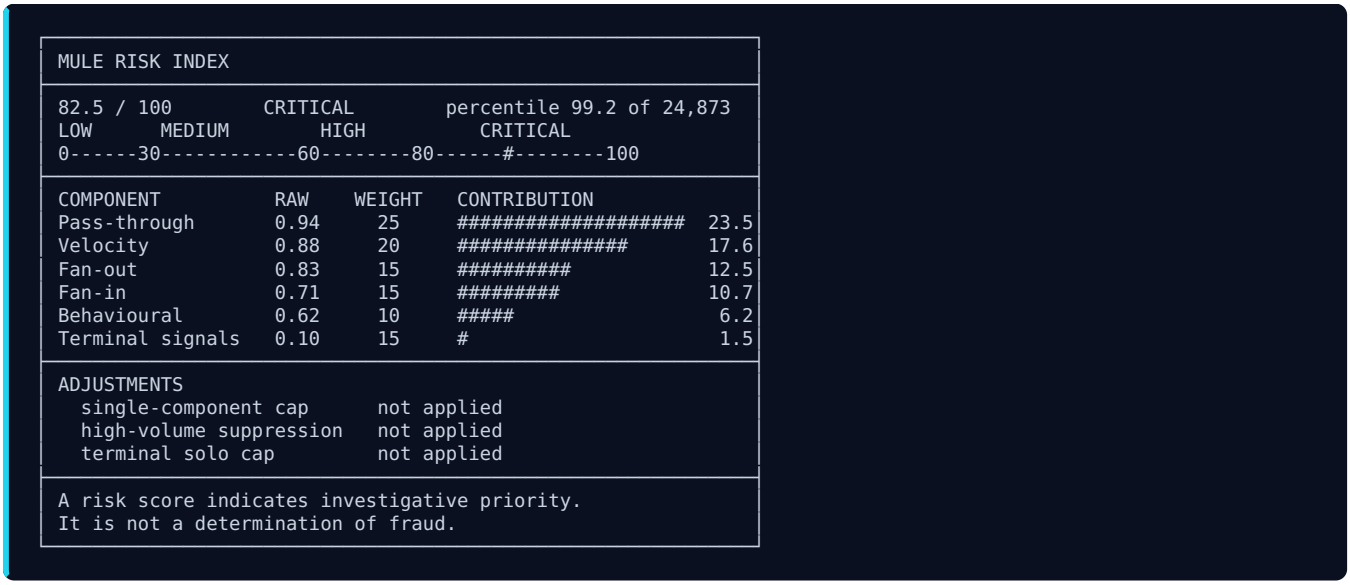


14.1 Rules

1. **Both the incoming and the outgoing block are rendered for every account, including when one side is zero** (then it reads `0 senders · ₹0 · 0 transactions`). Omitting an empty side teaches the wrong model. **MUST**
2. The role badge sits **beside** the flow panel, never above it as a title — the flow is the evidence, the role is the interpretation.
3. The role badge tooltip reads: "Role describes this account's position in this case, not a limit on what it can do. This account both receives and sends."
4. The same account appearing in two cases with two different layers is correct and must not be "fixed". The case chip on the classification card makes the scope explicit.
5. Copy discipline: use "**received**" and "**forwarded/dispersed**", never "collected only" or "pure distributor".

---

15. Risk Score UI



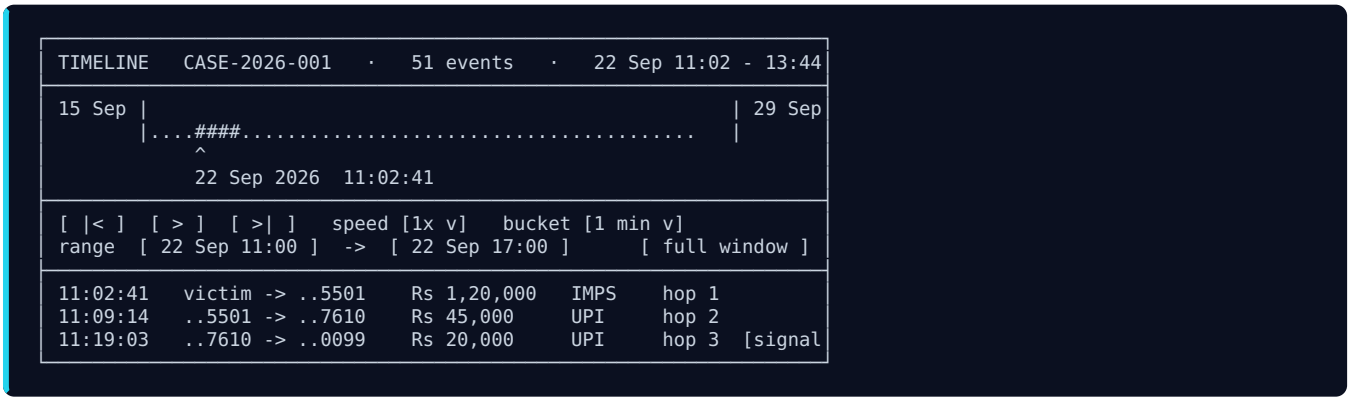
15.1 Field map — `GET /api/risk/{account_no}`

Element	Source
Score + band	`mri`, `band`
Band scale	`band_edges` (**read from the response, never hard-coded**)
Percentile	`percentile`
Component rows	`components[*].raw`, `.weight`, `.weighted`
Adjustment rows	`adjustments[*]` — including those with `applied: false`
Reasons	`reasons[]`, verbatim
Disclaimer	`disclaimer`, verbatim
Regime note	`calibration_regime`

15.2 Rules

1. **The frontend MUST NOT calculate the MRI, any component, or any weight.** It renders six numbers and draws six bars. No `reduce`, no re-weighting, no normalising. **MUST**
2. Band thresholds come from `band_edges`. If `config.py` changes, the UI follows automatically. **[API]**
3. **Adjustments that did not fire are still listed**, greyed, as "not applied". Showing the guard rails is part of showing the work.
4. `terminal_signals` renders its `cooccurrence_discount` inline when < 1.0, with the tooltip: "These indicators co-occur on the same rows in this dataset, so they are counted once, not several times."
5. The `disclaimer` string is rendered verbatim and may not be hidden, collapsed, or restyled into invisibility. **[PS]**
6. `calibration_regime` is shown as a small caption (e.g. "heuristic self-validation"). A score without its regime is not publishable — on screen or on a slide.

16. Timeline



16.1 Requirements

#	Requirement	Implementation
1	**15-day range**	Track spans `profile.date_range`; the traced window is highlighted inside it
2	**Minute-level filtering**	`minute_bucket ∈ {1, 5, 15, 60}`; bucketing is **server-side** via `/api/timeline`
3	**Play / pause**	`requestAnimationFrame` clock; speeds 0.5× / 1× / 2× / 5× / 10×
4	**Drag slider**	Scrub handle with keyboard support (`←` / `→` one bucket, `Home` / `End`) [A11Y]
5	**Start / end time**	Two datetime inputs, clamped to the dataset window; `[full window]` resets
6	**Transaction event display**	The event list under the track, synchronised with the playhead

16.2 How playback drives the graph

```
playhead t
    │
    ├─ events with timestamp <= t   → edge style "revealed", node style "active"
    ├─ events with timestamp >  t   → edge style "pending" (20% opacity)
    └─ cumulative[] at t            → "₹1,65,000 traced so far · 7 accounts touched"
```

No refetch during playback. The entire event array for the case window is already in memory (capped at `TIMELINE_MAX_EVENTS` = 5,000); playback is a style update over existing Cytoscape elements, batched per frame. **MUST**

16.3 Rules

1. The frontend owns only **filter UI state and the playhead**; it requests the appropriate data and renders what comes back. It does not aggregate, re-bucket or interpolate events. **MUST**
2. The histogram uses `buckets[]` from the API, **not** a client-side group-by of `events[]` — those differ when the event list is truncated, and the API's buckets are the accurate ones. **[API]**
3. Changing the bucket size or the time range issues a new request (debounced 250 ms); changing the playhead never does.
4. `stats.truncated` shows a caption: "Showing the first 5,000 events — the histogram above covers all of them."
5. Clicking any event selects the corresponding edge and opens the Edge Inspector; clicking an account name selects the node.

17. Transaction Details — the Edge Inspector

Clicking a graph edge, a timeline event or a transaction-table row opens:



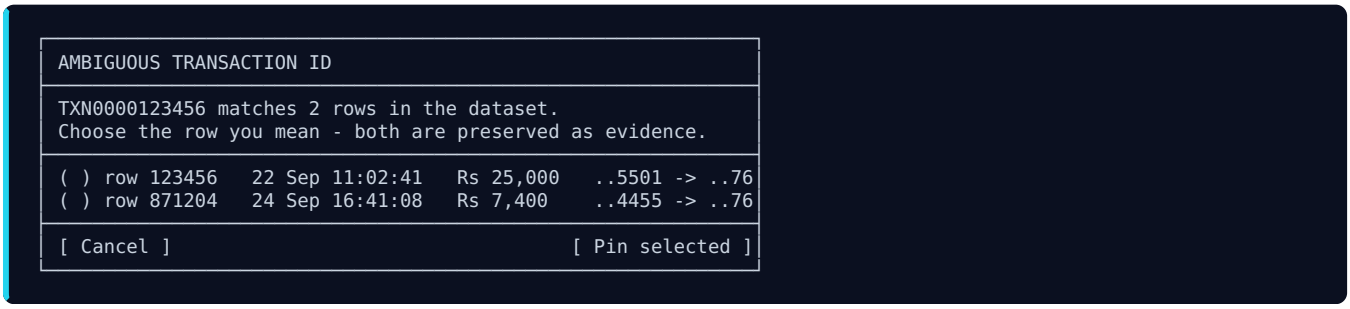
17.1 Required fields

Field	Source	Display
Transaction ID	`transaction_id`	Mono; a **duplicated** badge when `transaction_id_is_duplicated`
Source Row ID	`source_row_id`	**Always visible in this view**, mono, with copy and verify actions
Sender / Receiver	`sender_account` / `receiver_account`	Masked per setting, click to select that node
Amount	`amount`	`formatCurrency`, large
Timestamp	`timestamp`	`22 Sep 2026, 11:02:41`
Payment mode	`payment_mode`	Badge
Narration	`narration`	Verbatim, monospace — never reinterpreted
IP	`ip_address`	Octets partially masked by default; `[foreign IP]` chip when the row carries that signal
Device	`device_type`	`[automation]` chip for `Web_Emulator` / `Linux_Script`
Sender / Receiver IFSC	`sender_ifsc` / `receiver_ifsc`	Mono

17.2 Rules

1. **`source_row_id` is visible in the transaction detail and the evidence view**, because `Transaction_ID` may not be unique. Pinning, citing and verifying all key on the row id. **MUST [API]**
2. `[ verify ]` calls `GET /api/rows/{source_row_id}` and shows the raw CSV field values beside the interpreted ones. This is the "prove it" button, and it is the one a sceptical judge will press.
3. A signal chip is a **neutral** chip with an evidence tooltip, never a red "FRAUD" tag. A row with `Linux_Script` is a row with an automation device, nothing more. **[PS]**
4. **Multiple rows on one edge** are listed individually with their own row ids and amounts; the edge total is the backend's `amount`, not a client-side sum.

5. When an action requires a single row but the chosen `Transaction_ID` resolves to several, the UI must render the disambiguation dialog rather than guessing:



This dialog is triggered by the backend's `AMBIGUOUS_TRANSACTION_ID` (409) or by `skipped[].reason` on a basket add. **[API]**

---

18. Evidence Basket



18.1 What can be pinned

Pinnable	Sent as	Where the button lives
Account	`account_ids[]`	Node inspector, account page, search results
Transaction	`source_row_ids[]` (preferred) or `transaction_ids[]`	Edge inspector, transaction table, timeline event
Graph relationship	`edge_ids[]`	Edge inspector
Suspicious signal	the `source_row_id` carrying it	Terminal-evidence card
Timeline event	the `source_row_id` of the event	Timeline event row

Everything is normalised to `source_row_id` by the backend. The frontend prefers to send row ids directly, which avoids the ambiguity path entirely. **[API]**

18.2 Interaction rules

1. `[ + ADD TO EVIDENCE ]` appears in **every** context where an evidence-bearing object is displayed; it is never buried in a menu.
2. Optimistic add: the item appears immediately, then reconciles with `items_added` / `skipped`. A skipped item animates out with a toast naming the reason (ambiguous ID, duplicate pin, not found).
3. The basket badge in the sidebar shows the live item count and is `aria-live="polite"`. [A11Y]
4. Removing an item calls `DELETE …/{item_id}` and shows "Removed from basket. The source record is unchanged." — **the UI must make clear that un-pinning never deletes data**. **MUST**
5. The verification line (`verification.all_rows_reverified`) is shown; if any item fails re-verification, it is flagged in place and the document buttons are disabled with the reason.
6. Document buttons read `case.readiness.blockers` and render disabled with the blocker text as the tooltip — the UI never offers an action the backend will reject. **[API]**

19. Case Management

Route `/cases` · `pages/CaseList.jsx`

Column	Source	Notes
Case ID	`case_id`	Mono, primary link
Victim Account	`victim_account`	Masked
Created At	`created_at`	Relative + absolute on hover
Status	`status`	`created` · `traced` · `evidence_collected` · `documented` as a stepper chip
Risk Level	highest `mri` among the case's traced accounts, as returned	Band badge with the numeric score
Evidence Items	`evidence_basket.item_count`	
Document Status	derived from `documents[]`	`—` · `Diary` · `Diary + Freeze`

Rules

1. Rows are keyboard-navigable; `Enter` opens `/cases/{case_id}`. [A11Y]
2. Cases created against a different `dataset_fingerprint` are shown with a **read-only** chip and an explanatory tooltip. **[API]**
3. Empty list → `EmptyState` "No cases yet. Start an investigation from the dashboard." with the action button.
4. The list is the demo's recovery path: if anything breaks mid-presentation, the previous case is one click away.

---

20. Case Page

Route `/cases/:caseId` · `pages/CaseDetail.jsx`

One call rehydrates everything: `GET /api/cases/{case_id}`. **[API]**

Section	Content	Source
Overview	Victim, incident time, status, FIR/officer fields (editable), dataset fingerprint, readiness blockers	`case`, `readiness`
Money Trail	The trace summary and a button to reopen the graph at `/investigation?case=…&trace=…`	`trace`
Accounts	Every account in the trace with case-relative layer, role, confidence, risk, traced value	`layers`, `risk_scores`
Transactions	The rows cited in this case, with `source_row_id`	basket + trace edges
Evidence	The basket (section 18), inline	`evidence_basket`
Timeline	Static summary + a link into the playback view	`timeline`
Documents	Generated documents with status, validation result and download	`documents`

The purpose of this page is to return to an investigation without re-running the trace. Opening a case **MUST NOT** issue `POST /api/trace`; it uses the stored `trace_id`. Re-tracing is an explicit button labelled "Re-run trace with current settings". **MUST**

Administrative edits (`fir_number`, `investigating_officer`, `notes`) `PATCH` the case and immediately refresh `readiness`, so the freeze-requisition blockers clear in front of the user.

21. Documents Page

Route `/documents` · `pages/Documents.jsx`

Document	Status values	Actions
Case Diary	`Not Generated` · `Generating` · `Generated` · `Failed`	Generate · Open PDF · Download · Regenerate
Freeze Requisition	same	Select accounts · Generate · Open PDF · Download

21.1 Status semantics

Status	Shown when	UI
`Not Generated`	No document of this kind for the case	Primary button enabled if `readiness` allows
`Generating`	Request in flight	Button disabled, inline progress, elapsed timer (budget 10 s)
`Generated`	201 received	Green check, page count, generation time, **"47/47 facts verified"**
`Failed`	Error returned	Red state with the backend's reason; for `EVIDENCE_VALIDATION_FAILED`, the failing facts are listed

21.2 Rules

1. **The frontend never creates legal facts.** It sends a format, a section list and a selection of accounts. Every sentence, number and citation in the output comes from the backend's validated EvidenceObject. **MUST [PS]**
2. On `EVIDENCE_VALIDATION_FAILED` the UI states plainly: "No document was generated." and lists `details.failures` (validator, field path, expected, found, row id). It must never imply a partial document exists. **[API]**
3. `[ Open PDF ]` opens `GET /api/documents/{document_id}` in a new tab; `[ Download ]` uses the same URL with a download attribute. The frontend never constructs a file path — only the opaque `document_id`. **MUST**
4. The validation summary ("47/47 facts verified", validators V1–V12) is displayed next to every generated document. It is the product's central claim, and it should be visible without clicking.
5. If `llm_available` is false and the user ticked prose, show the info toast "Local model unavailable — generated from templates, which is the fully supported path." Generation still succeeds. **[API]**

22. Case Diary UI

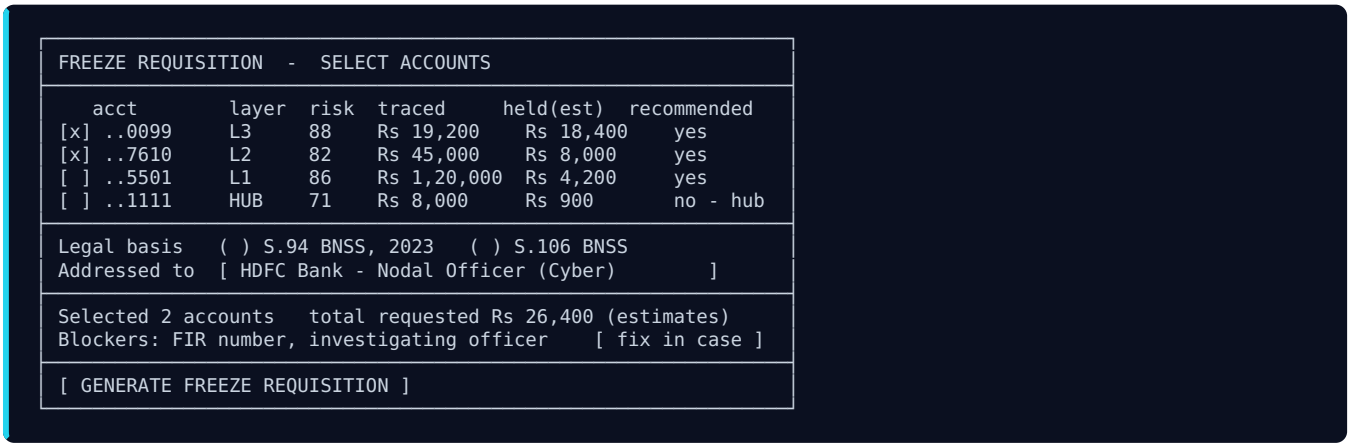


Rules

1. The pre-generation panel shows **counts from the basket and the case**, so the investigator knows exactly what will be included before a document exists.
2. "Verified Accounts" is `verification` data from the basket, not a client-side count of what looks fine.
3. After generation the panel shows page count, generation milliseconds and the validator result — all backend fields.
4. Regenerating creates a **new** `document_id`; previous documents remain listed and downloadable. Evidence history is never overwritten. **[DEC]**

---

23. Freeze Requisition UI



Rules

1. **The investigator selects the accounts. The backend recommends; it does not decide.** `freeze_recommended` drives the default tick, but every checkbox is user-controlled. **[API]**
2. Accounts flagged `is_hub` are listed with the recommendation `no — hub` and are **unticked and disabled** by default; enabling one requires the explicit override, and the UI repeats the warning that freezing a payment aggregator is a serious false positive.
3. Amounts are labelled **traced** and **held (estimate)** — never "balance". The footer repeats the disclaimer that held values are dataset-derived.
4. Blockers (`fir_number`, `investigating_officer`) are shown with a direct `[ fix in case ]` link that opens the case overview fields inline.
5. The legal-basis selector renders the backend's citation string; the frontend never composes a legal reference of its own.

---

24. Dataset / System Page

Route `/system` · `pages/System.jsx`



24.1 Content

Block	Source
Backend, uptime, API version	`/api/health`
Dataset status card	`/api/ingest/profile` (section 5)
Graph / features loaded, memory	`/api/health`
Profile verification + mismatches	`profile.verification`
LLM availability	`health.llm_available`
Ground-truth availability	`health.ground_truth_available`
Ingestion / graph-build / feature timings	`/api/ingest/manifest.stage_timings_ms`
Benchmarks vs targets	`/api/benchmarks`
Ingest controls	`POST /api/ingest`, `/api/ingest/status/{job_id}`

24.2 Rules

1. **Do not fabricate numbers.** Every unmeasured benchmark renders as `–` with the caption "not yet measured". There is no placeholder, no "~", no number borrowed from another machine. **MUST [PS]**
2. Measured values render beside their target with a pass/fail chip — the one place the product claims performance, and it claims only what was measured.
3. `LLM UNAVAILABLE` is presented **neutrally**, as a supported configuration, not as an error.
4. `Ground truth UNAVAILABLE` links to the evaluation explanation: metrics report a status rather than a score until labels exist. **[API]**
5. The ingest panel reuses the section-8 progress component with the backend's own nine stage names.

25. Search Experience

Global search (top bar, `/` to focus) resolves **accounts** and **cases**.



25.1 Behaviour

Aspect	Rule
Trigger	2+ digits, debounced **300 ms**; `Esc` clears; `↑/↓` + `Enter` select [A11Y]
Account route	`GET /api/accounts/search?q=…&limit=20&sort=risk_desc`
Case route	Client-side filter over the already-loaded case list (tiny)
Result row	Account (masked, mono) · **risk score + band** · **layer hint + role** · in/out counts · IFSC
No match	`results: []` renders "No account matches…" plus the backend's `suggestions` as "did you mean" chips — **never an error state** [API]
Exact 12-digit match	Shown first with an `[ INVESTIGATE ]` action inline, so the blind test is one keystroke away

25.2 Rules

1. Sorting and filtering are **query parameters**, not client-side array operations. The backend's `sort=risk_desc` is authoritative; a client-side re-sort of a truncated list would lie. **MUST**
2. Every result shows **both** in and out counts — reinforcing that roles are not capabilities (section 14).
3. The caption "layer/role shown are global hints, not case-relative classifications" is permanent, not dismissible. **[API]**
4. Search never auto-navigates. The investigator chooses.

26. API Service Layer

26.1 The single client

```
// services/api.js — the ONLY axios instance in the application
import axios from "axios";

export class ApiError extends Error {
  constructor(code, message, details = {}, requestId = null, status = null) {
    super(message); this.code = code; this.details = details;
    this.requestId = requestId; this.status = status;
  }
}

const client = axios.create({
  baseURL: "/api",                 // relative: works behind any proxy
  timeout: 30_000,
  headers: { "Content-Type": "application/json" },
});

// Unwrap the standard envelope: {success, data, meta}
client.interceptors.response.use(
  (res) => {
    const { success, data, meta } = res.data ?? {};
    if (success === false) {
      const e = res.data.error ?? {};
      throw new ApiError(e.code ?? "INTERNAL_ERROR", e.message, e.details,
                         meta?.request_id, res.status);
    }
    if (meta?.warnings?.length) warningBus.emit(meta.warnings);
    if (meta?.dataset_fingerprint) fingerprintBus.check(meta.dataset_fingerprint);
    return Object.assign(data ?? {}, { __meta: meta ?? {} });   // timings available
  },
  (err) => {
    if (axios.isCancel(err)) throw err;
    const body = err.response?.data;
    if (body?.error) {
      throw new ApiError(body.error.code, body.error.message, body.error.details,
                         body.meta?.request_id, err.response.status);
    }
    throw new ApiError("BACKEND_OFFLINE",
                       "Cannot reach the investigation engine.", {},
                       null, err.response?.status ?? 0);
  }
);

export default client;
```

What the interceptor buys: every component receives plain `data`, every error is an `ApiError` with a code from the closed set, `meta.processing_ms` is available as `__meta` for the latency readouts, backend warnings surface automatically, and a dataset swap is detected centrally.

26.2 Service functions

```
// services/health.js
export const health            = ()             => api.get("/health");

// services/ingest.js
export const ingest            = (form)         => api.post("/ingest", form,
                                     { headers: { "Content-Type": "multipart/form-data" } });
export const getIngestStatus   = (jobId)        => api.get(`/ingest/status/${jobId}`);
export const getDatasetProfile = ()             => api.get("/ingest/profile");
export const getManifest       = ()             => api.get("/ingest/manifest");

// services/accounts.js
export const searchAccounts    = (q, opts)      => api.get("/accounts/search", { params: { q, ...opts } });
export const accountExists     = (no)           => api.get(`/accounts/${no}/exists`);
export const getAccount        = (no, opts)     => api.get(`/accounts/${no}`, { params: opts });
export const getTransactions   = (no, params)   => api.get(`/accounts/${no}/transactions`, { params });
export const getClassification = (no)           => api.get(`/accounts/${no}/classification`);
export const getTerminalEvidence = (no)         => api.get(`/accounts/${no}/terminal-evidence`);
export const getTransactionById  = (id)         => api.get(`/transactions/${id}`);   // returns a LIST
export const getRow            = (rowId)        => api.get(`/rows/${rowId}`);

// services/cases.js
export const createCase        = (body)         => api.post("/cases", body);
export const listCases         = ()             => api.get("/cases");
export const getCase           = (id)           => api.get(`/cases/${id}`);
export const patchCase         = (id, body)     => api.patch(`/cases/${id}`, body);
export const getCaseClassification = (id, no)   => api.get(`/cases/${id}/accounts/${no}/classification`);

// services/trace.js
export const traceVictim       = (body, signal) => api.post("/trace", body, { signal });
export const getTrace          = (traceId)      => api.get(`/trace/${traceId}`);

// services/graph.js
export const getGraph          = (no, params)   => api.get(`/graph/${no}`, { params });
export const expandNode        = (no, params)   => api.get(`/graph/${no}/expand`, { params });

// services/timeline.js
export const getTimeline       = (params)       => api.get("/timeline", { params });

// services/risk.js
export const getRisk           = (no)           => api.get(`/risk/${no}`);
export const getRiskBatch      = (accountNos)   => api.post("/risk/batch", { account_nos: accountNos });
export const getRiskDistribution = ()           => api.get("/risk/distribution");

// services/evidence.js
export const addEvidence       = (body)         => api.post("/evidence/basket", body);
export const getEvidence       = (caseId)       => api.get(`/evidence/basket/${caseId}`);
export const removeEvidenceItem = (caseId, itemId) => api.delete(`/evidence/basket/${caseId}/${itemId}`);

// services/documents.js
export const generateCaseDiary        = (caseId, body) =>
  api.post(`/cases/${caseId}/documents/case-diary`, body);
export const generateFreezeRequisition = (caseId, body) =>
  api.post(`/cases/${caseId}/documents/freeze-requisition`, body);
export const documentUrl       = (docId)        => `/api/documents/${docId}`;   // href, not fetch
export const getDocumentMeta   = (docId)        => api.get(`/documents/${docId}/meta`);
```

26.3 Rules

1. **Components and pages must not contain raw `fetch` / `axios` calls.** They call hooks; hooks call services. A lint rule (`no-restricted-imports` for `axios` outside `src/services`) enforces it. **MUST**
2. One function per endpoint, named for the action, typed against `types/api.d.ts`.
3. Downloads are **hrefs**, not fetches — `documentUrl()` returns a URL for an `<a download>`; the app never buffers a PDF in memory.
4. Every long-running call accepts an `AbortController` signal so navigation and `[ Cancel ]` actually cancel.
5. Retry policy: **GET** idempotent reads retry once on a network error with 500 ms backoff; **POST** never auto-retries (a double trace is wasteful, a double case is wrong).

---

27. State Management

27.1 Shape of the global investigation state

```
const initialState = {
  // identity
  currentCase:   null,   // {case_id, victim_account, status, readiness}
  victimAccount: null,   // "551100220033"

  // results (bounded payloads only)
  traceResult:   null,   // nodes[], edges[], summary, performance, parameters_used
  graphData:     null,   // nodes[], edges[], stats  (<= 800 / 1500)
  timeline:      null,   // events[] (<= 5000), buckets[], cumulative[]

  // selection
  selectedNode:  null,   // account_no
  selectedEdge:  null,   // edge_id
  playhead:      null,   // ISO timestamp

  // per-account detail, LRU-capped
  accountCache:  new LRU(50),   // account_no -> {account, risk, classification}
  risk:          null,          // the selected account's risk payload

  // evidence
  evidence:      { basketId: null, items: [], counts: {}, verified: true },

  // process
  loading:       { stage: "idle", since: null },
  timings:       { trace: null, graph: null, evidence: null },
  errors:        null,   // {code, message, details, requestId}
  warnings:      [],     // from meta.warnings
};
```

27.2 Rules

#	Rule
1	**Never store the 2 M-row dataset, or any unbounded collection, in the browser.** Everything in state is a bounded API response **MUST**
2	`accountCache` is an **LRU of 50**, keyed `(dataset_fingerprint, account_no)`. A fingerprint change clears it entirely
3	Transaction tables are **not** in global state — they are page-local, paginated and discarded on unmount
4	Selection state holds **ids**, never object copies, so there is exactly one source for each record
5	Derived display values are computed in `useMemo` at render time; they are never persisted into state (that is how two numbers start disagreeing)
6	The reducer is the only writer; components dispatch intents (`SELECT_NODE`), not state patches
7	State is mirrored into the URL for the five fields that matter (`account` , `case` , `trace` , `node` , `t`) and rehydrated from it on load
8	No analytical field is ever written by the client — risk, layer, role and confidence are read-only values that arrived from the backend

28. Performance Requirements

28.1 Hard prohibitions

Never	Instead
Load 2 M rows into React	Paginate: `page_size ≤ 500`, server-side filters
Render 2 M graph edges	Render the bounded subgraph (≤ 800 / 1,500)
Compute graph traversal in the browser	`POST /api/trace` — the CSR lives in the backend
Re-request the same account repeatedly	LRU cache keyed by fingerprint + account
Re-render the whole graph on selection	Cytoscape class toggles on the affected elements only

28.2 Required techniques

Technique	Applied to
Memoization	`React.memo` on inspector, summary tiles, legend, table rows; `useMemo` for derived display objects
Virtualization	Transaction tables, search results beyond 50 rows, the timeline event list
Lazy loading	Route-level `React.lazy` for `/cases`, `/documents`, `/system`; terminal-evidence section loads on expand
Debouncing	Search 300 ms, time-range change 250 ms, node-click detail fan-out 300 ms
Incremental graph updates	`cy.batch()` for add/patch; never `cy.json()` wholesale replacement
Pagination	Every list that can exceed 100 rows
Request cancellation	`AbortController` on navigation and on a superseded search

28.3 Budgets

Interaction	Budget	Measured how
First meaningful paint (shell + health)	1.0 s	Manual, on the demo laptop
Dashboard fully populated	1.5 s	
Blind query → graph on screen	**≤ 3.0 s** (of which ≤ 2.0 s is backend trace [PS])	`performance.total_ms` + render time
Node selection → inspector populated	300 ms	Cached: instant
Graph pan/zoom at 500 nodes	≥ 30 fps	Devtools frame timing
Timeline playback	60 fps at 1×, no dropped frames at 5×	

29. Graph Performance

29.1 Cytoscape configuration

```
const cy = cytoscape({
  container,
  elements: [],                  // added in one batch after mount
  style: cytoscapeStyles,        // static stylesheet; no per-render style objects
  layout: { name: "preset" },    // layout is run explicitly, never implicitly
  textureOnViewport: true,       // fast pan/zoom at high element counts
  hideEdgesOnViewport: true,
  motionBlur: false,
  pixelRatio: 1,                 // 1 is enough on a projector and much cheaper
  wheelSensitivity: 0.2,
  maxZoom: 4, minZoom: 0.2,
});

// one layout run per data change — NOT per selection
function applyLayout(elementCount) {
  const name = elementCount > 400 ? "breadthfirst" : "dagre";
  cy.layout({ name, rankDir: "TB", nodeDimensionsIncludeLabels: false,
              animate: false, fit: true, padding: 24 }).run();
}
```

Setting	Why
`dagre` (`rankDir: TB`) default	Layered money flow reads top-to-bottom; hop depth becomes visual
`breadthfirst` above ~400 elements	Deterministic and much cheaper than force-directed at scale
`cose` fallback	Only when dagre is unavailable or the graph is non-hierarchical (rings)
`animate: false`	A 500-node animated layout is the single most likely demo freeze
`textureOnViewport`	Keeps pan/zoom smooth at 1,500 edges

29.2 Rules

1. **Never run a layout on selection.** Selection toggles classes: `js cy.batch(() => { cy.elements(".selected").removeClass("selected"); cy.$id(accountNo).addClass("selected").neighborhood().addClass("adjacent"); });` **MUST**
2. Expansion **patches**: `cy.add(newElements)` then a layout restricted to the added subtree (`eles: added`). The existing positions do not move, so the investigator does not lose their mental map.
3. Playback updates styles per frame inside a single `cy.batch()`; no element is added or removed during playback.
4. Element count is capped client-side as a second line of defence; if a response somehow exceeds the cap, the UI renders the cap and shows a warning rather than attempting it.
5. Node labels are plain strings resolved at add time — no label functions, which re-evaluate on every repaint.
6. The Cytoscape instance lives in a ref inside `useGraph`, is created once per mount and destroyed on unmount. It is **never** stored in React state.

30. Error States

Every error is an `ApiError` with a code from the backend's closed set. `utils/errorCopy.js` maps code → investigator-facing copy. **No raw stack traces, no JSON dumps, no Axios messages reach the user. MUST**

Code	Title	Body	Primary action
`ACCOUNT_NOT_FOUND`	Account not found	"Account not found in the current dataset." + "did you mean" chips from `details.suggestions`	Retry with a suggestion
`DATASET_NOT_LOADED`	Engine not ready	"The investigation engine is not ready. No dataset has been indexed yet."	Go to System
`INGESTION_IN_PROGRESS`	Indexing	"The dataset is still being indexed — {progress}% complete."	Watch progress
`TRACE_LIMIT_EXCEEDED`	Investigation too large	"The requested investigation is too large. Narrow the time range or reduce the scope."	Apply the suggested bound
`GRAPH_LIMIT_EXCEEDED`	View too large	"This view exceeds the display limit. Reduce the radius or expand nodes individually."	Set radius = 1
`AMBIGUOUS_TRANSACTION_ID`	Which record?	"This transaction ID matches more than one record."	Open the row picker (17.2)
`EVIDENCE_VALIDATION_FAILED`	Not generated	"Evidence validation failed. **No document was generated.**" + the failing facts	Review evidence
`EVIDENCE_BASKET_EMPTY`	Nothing pinned	"Pin at least one account or transaction before generating a document."	Go to the graph
`CASE_NOT_FOUND`	Case not found	"That case no longer exists."	Back to cases
`DOCUMENT_GENERATION_FAILED`	Generation failed	"The document could not be produced." + `details.stage`	Retry as Markdown
`DOCUMENT_NOT_FOUND` / `DOCUMENT_EXPIRED`	File unavailable	"This document is no longer available."	Regenerate
`INVALID_TIME_RANGE`	Invalid dates	"Select a range inside {dataset_window}."	Reset to full window
`INVALID_ACCOUNT`	Invalid format	"Account numbers are 12 digits."	Focus the field
`GROUND_TRUTH_UNAVAILABLE`	(not an error)	"No labelled ground truth is available for this dataset, so detection metrics are not reported."	Open the explanation
`PROFILE_MISMATCH`	(warning banner)	"The dataset profile disagrees with the expected values." + the mismatch list	Open System
`BACKEND_OFFLINE`	Backend offline	"Cannot reach the investigation engine. Retrying…"	Retry now
`INTERNAL_ERROR`	Something went wrong	"Something went wrong while processing the investigation." + `Reference: {request_id}`	Retry

30.1 Presentation rules

1. **Scope matters:** a field error renders inline; a panel error renders inside that panel; only a failure that invalidates the whole page takes the page. A failed timeline must not blank a good graph. **MUST**

2. Every error state shows **one** primary action. A dead end is a defect.
3. `meta.request_id` is always displayed in small type on `INTERNAL_ERROR` — that one string turns "it broke" into a one-grep diagnosis.
4. Errors are announced with `role="alert"`. [A11Y]
5. The two non-errors (`GROUND_TRUTH_UNAVAILABLE`, `PROFILE_MISMATCH`) are rendered as **informational states**, never as red failures. They are honest answers. **[API]**

---

31. Empty States

Every empty state names the situation, explains it in one line, and offers the next action.

Situation	Message	Action
No investigation yet	"Enter a victim account to begin investigation."	Focus the search field
Trace returned nothing	"No outbound flow from this account in the selected window. The account may be a mule rather than a victim."	**[Trace upstream instead]** · [Widen window]
No transactions in range	"No transactions found for this account in the selected time range."	[Clear filters]
No terminal endpoint	"No terminal endpoint identified in the current trace. Terminal indicators appear on roughly 0.13% of rows and cannot identify every layer."	[View L3 candidates]
No evidence pinned	"No evidence has been pinned yet. Select an account or transaction and choose **Add to Evidence**."	[Open the graph]
No cases	"No cases yet. Start an investigation from the dashboard."	[New investigation]
No documents	"No documents generated for this case."	[Generate case diary]
Search no match	"No account matches that number." + suggestions	Chips
Evaluation unlabelled	"Detection metrics are unavailable because no ground-truth labels exist for this dataset."	[What does this mean?]

The empty trace is the one that matters. A blind test ID that turns out to be a mule rather than a victim is a realistic scenario, and the upstream fallback is the difference between "our system found nothing" and "our system found who funded it". It is one click. **[PS]**

32. Responsive Design

Primary target: desktop / laptop. Minimum supported: 1280 × 720. This is an investigator workstation application; mobile is explicitly not prioritised. **[DEC]**

Breakpoint	Layout
≥ 1920	Sidebar expanded · graph + 420 px inspector · timeline 200 px
1440–1919	Sidebar expanded · graph + 380 px inspector · timeline 180 px
1280–1439	Sidebar collapses to a 56 px icon rail · inspector becomes an overlay drawer over the graph's right third · timeline 160 px, collapsible
1024–1279	Inspector stacks **below** the graph · timeline collapses to a summary strip that expands on demand
< 1024	Single column: search → summary → graph (min 360 px) → inspector. A one-time notice recommends a larger display. Nothing is removed

Rules

1. The graph always receives the largest available region at every breakpoint. **MUST**
2. Search and the system status are reachable at every width — they are never inside a collapsed menu.
3. No feature is removed on small screens; layout changes, capability does not.
4. The app must survive a projector at 1280 × 720: minimum body text 13 px, minimum graph label 11 px, and the demo checklist includes a projector rehearsal.

33. Accessibility

#	Requirement	Implementation
1	**Contrast**	Body text ≥ 7:1, secondary ≥ 4.5:1, graph labels ≥ 4.5:1 against the canvas. Verified with a contrast checker, not by eye
2	**Keyboard operable**	Every action reachable by `Tab`; graph nodes form a focusable ring (`Tab` through them, `Enter` selects); no mouse-only interaction exists
3	**Focus states**	2 px accent outline with 2 px offset, never `outline: none`
4	**Semantic buttons**	`<button>` for actions, `<a>` for navigation. No clickable `<div>`
5	**Tooltips**	Every graph icon, badge and abbreviation has a tooltip reachable by keyboard focus, not hover alone
6	**Font sizes**	Body 14 px, dense tables 13 px, never below 11 px; `rem`-based and respecting browser zoom to 200%
7	**Non-colour indicators**	**Mandatory** — see below
8	Live regions	`aria-live="polite"` for status, basket count and stage progress; `role="alert"` for errors
9	Reduced motion	`prefers-reduced-motion` disables playback easing and layout animation
10	Screen-reader graph fallback	The graph has an accessible sibling table (account, layer, role, risk, traced value, hop) — the same data, linearised

33.1 Never colour alone

Meaning	Colour	shape	text	number
Critical risk	red	diamond / thick border	`CRITICAL`	`91`
High risk	amber	thick border	`HIGH`	`78`
Medium	yellow	medium border	`MEDIUM`	`54`
Low	neutral	thin border	`LOW`	`12`
L1 / L2 / L3	distinct hues	ellipse / ellipse / diamond	`L1` `L2` `L3` labels	hop number
Terminal evidence	red marker	corner glyph	"terminal indicators"	signal count

Red-green colour blindness affects roughly 8% of men; a judge, an officer, or a teammate may be among them. A risk badge that reads `CRITICAL 91` with a distinct shape works for everyone. [A11Y]

34. Visual Language

34.1 Tokens

```
:root {
  /* surfaces — dark neutral, not black, not navy-neon */
  --bg-0:#0d1117; --bg-1:#141b24; --bg-2:#1b232e; --bg-3:#232d3a;
  --border:#2b3644; --border-strong:#3a4858;

  /* text */
  --text-0:#e8edf3; --text-1:#aab6c4; --text-2:#7a8798; --text-inv:#0d1117;

  /* one interaction accent */
  --accent:#2f81f7; --accent-hover:#4b95ff; --accent-soft:#122a4d;

  /* data semantics */
  --critical:#e5484d; --warn:#f0883e; --caution:#d9c05a;
  --ok:#3fb950; --info:#58a6ff; --muted:#6e7b8b;

  /* layers */
  --victim:#2f81f7; --l1:#d4a017; --l2:#f0883e; --l3:#e5484d;
  --neutral:#6e7b8b; --hub:#8b5cf6;

  /* type */
  --font-ui:"Inter",system-ui,sans-serif;
  --font-mono:"JetBrains Mono","SF Mono",ui-monospace,monospace;
  --fs-body:14px; --fs-dense:13px; --fs-label:11px;

  /* geometry — restrained */
  --radius:6px; --space:8px; --dur:150ms;
}
```

A light theme is optional and must pass the same contrast rules; the dark theme is the default because the graph canvas reads better and the demo room is dark. **[DEC]**

34.2 Usage rules

#	Rule
1	`--accent` **is for interaction only** (focus, selection, primary buttons, links). It never means "risky"
2	**Semantic colour is reserved for data.** Red is risk, never decoration
3	**All numbers are** `--font-mono` **with tabular figures.** Amounts, IDs, scores and timestamps line up in columns
4	Maximum **two** font weights per screen (400 / 600)
5	No gradients, no glows, no glass-morphism, no scanlines, no animated backgrounds
6	Borders over shadows; one elevation level for overlays only
7	8 px spacing grid; dense tables may use 4 px
8	Icons are single-colour line icons at 16/20 px, always paired with text in primary navigation

Reference feel: a cybercrime investigation console crossed with a financial-forensics terminal — Bloomberg-terminal density, government-report seriousness. **Not** a gaming dashboard, and not a consumer fintech app.

35. Security UX — observed vs inferred vs assessed

The interface must never blur the line between what the data **says**, what the system **infers**, and what that **means for priority**. Three labelled registers:

```
402938475610
OBSERVED      Device_Type = Linux_Script on 3 of 184 rows
              IP 194.5.x.x on the same 3 rows
              source rows 1884422, 1884510, 1884677
INFERRED      Terminal indicators present. In this dataset
              these indicators co-occur, so they count as
              one piece of evidence, not three.
ASSESSED      Terminal component contributes 1.5 of 82.5
              MRI points. Risk reflects investigative
              priority, not a determination of fraud.
```

35.1 The three registers

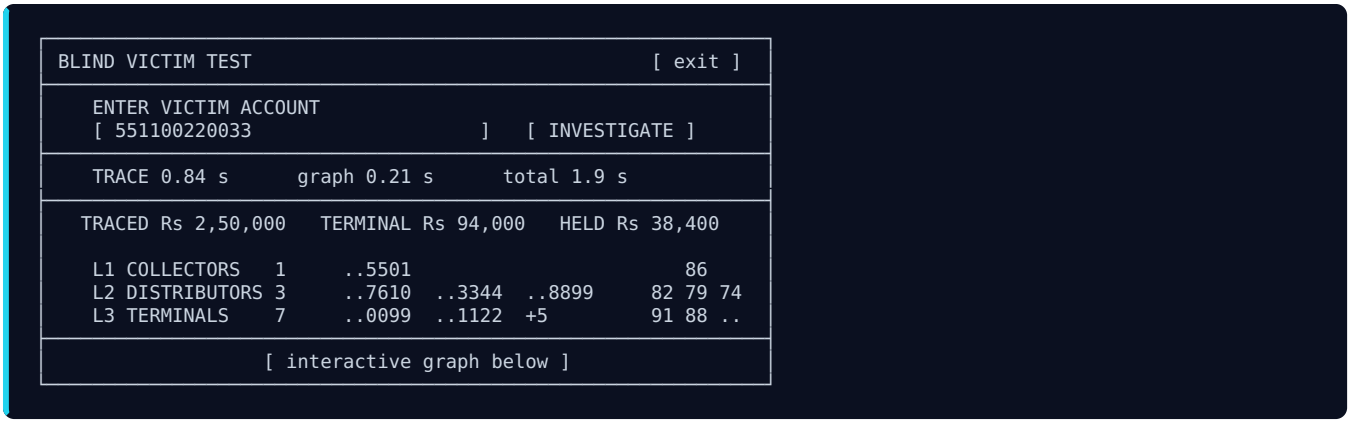
Register	Definition	Source	Visual treatment
OBSERVED	A value that exists in the source row	`/api/rows/{id}`, transaction fields	Monospace, neutral chip, `source_row_id` attached
INFERRED	A derived signal or classification	`terminal-evidence`, `classification`	Normal text, amber marker, always with confidence and reasons
ASSESSED	Priority expressed as a score	`/api/risk/{no}`	Score + band + the backend's disclaimer

35.2 Rules

1. **Never display a verdict.** Forbidden strings anywhere in the UI: "criminal", "fraudster", "guilty", "confirmed mule", "proven". The system identifies suspicious behaviour and evidence — **not guilt**. **MUST [PS]**
2. Every inference carries its evidence within one click: the terminal card lists the exact rows, the classification lists its ten inputs and reasons.
3. The co-occurrence caveat is shown wherever terminal signals appear: "these indicators co-occur in this dataset and are counted once, not several times."
4. The risk disclaimer (`risk.disclaimer`) is rendered verbatim beside every score. Tooltip-only placement is not acceptable on the node inspector or the account page.
5. `ProvenanceChip` is a shared component: any observed value may be wrapped in it to expose `source_row_id` + a verify action. Use it liberally — it is the cheapest credibility in the product.
6. Masked display mode (`?mask=true`) is available for screenshots and slides; the unmasked value stays one click away for the investigator.

36. Blind Victim Test Mode

This is the single most important workflow in the product. The official blind-victim test carries **40% of the judging weight** [PS]. A judge types an account number nobody on the team has seen and expects an answer immediately.



36.1 The interaction, in full

```
1. Judge types a 12-digit account   (field is already focused)
2. Judge presses Enter              (or clicks INVESTIGATE)
3. Named stages appear              (section 8 — real stages, real timings)
4. Results appear:  trace time · L1 · L2 · L3 · traced value · graph
5. Judge clicks any node            → inspector with layer, role, risk, reasons

TOTAL CLICKS BEFORE AN ANSWER: 1  (zero if they press Enter)
```

36.2 Requirements

#	Requirement
1	Reachable from anywhere: a **Blind Test** item in the sidebar and the `b` shortcut
2	The input is focused on entry. `Enter` runs the whole sequence. **No dialogs, no confirmations, no wizard**
3	The six required outputs appear together as soon as the trace resolves: trace time, L1 / L2 / L3 accounts, total traced value, and the interactive graph
4	Layer lists show the account and its risk score, and each entry is clickable straight into the inspector
5	The **upstream fallback** is one click when the ID turns out not to be a victim: "No outbound flow — [Trace upstream instead]"
6	Trace time is displayed prominently; it is the performance claim being judged
7	Nothing on this screen is precomputed, cached from a rehearsal, or seeded. It runs the same endpoints as the normal flow
8	A failure degrades visibly and usefully: the error copy from section 30 with one retry action — never a blank screen

36.3 What this mode removes

Case creation prompts, incident-time pickers, evidence affordances, document buttons, administrative fields. They remain available via `[ Promote to full case ]`, which opens the same investigation in the standard page with the case already created. The judge sees a focused instrument; the investigator keeps the full tool.

37. Demo Mode

A toggle in the top bar (`SystemContext.demoMode`, persisted to `localStorage`).

37.1 What demo mode may do

Allowed	Implementation
Preload a known dataset	Trigger `POST /api/ingest` with the standard path, or detect it already loaded
Offer known starting points	Five pre-selected victim account numbers as clickable chips
Preload a known case	`GET /api/cases/{id}` for a case created earlier during the same session
Show the actual investigation	Identical code path, identical endpoints, identical results
Reset the UI	Clear selection, playhead, filters and caches; return to the dashboard in one click

37.2 What demo mode must never do

Forbidden	Why
Fabricate results	Everything displayed comes from the live backend and the real dataset. **MUST**
Replay a recorded response	A stubbed payload is a lie told at the worst possible moment
Fake timings	The latency readouts remain the backend's measured values
Hide failures	If the backend errors in demo mode, the error shows
Alter scores, layers or reasons	Not even "for clarity"

Demo mode changes what is on screen first, never what is true. It is a bookmark bar, not a stage set. [PS-adjacent]

The one-click **[Reset demo]** action is a genuine deliverable: after a judge has pulled the graph apart, the next conversation starts clean without a page reload.

---

38. Frontend Routes

Route	Page	Purpose	Guard
`/`	`Dashboard`	System overview + immediate investigation	Always available
`/investigation`	`Investigation`	The investigation workspace	Requires `dataset_loaded`; otherwise an "engine not ready" state
`/cases`	`CaseList`	All cases	Requires dataset
`/cases/:caseId`	`CaseDetail`	One case, rehydrated	404 state on `CASE_NOT_FOUND`
`/accounts/:accountId`	`AccountDetails`	Full account view outside a case	404 state on `ACCOUNT_NOT_FOUND`
`/evidence`	`Evidence`	Basket across the active case	Requires an active case
`/documents`	`Documents`	Generate and download documents	Requires an active case
`/system`	`System`	Dataset, profile, ingest, benchmarks	**Always available** — it is the recovery route
`/blind`	`Investigation` (blind variant)	Section 36	Requires dataset

Query parameters carry investigation state (`account`, `case`, `trace`, `node`, `t`, `autostart`), so any screen is linkable and reload-safe (section 6.4).

Unknown routes render a 404 page with navigation — never a blank screen.

39. Component List

39.1 Core

Component	Responsibility	Primary data
`AppShell`	Grid, providers, keyboard shortcuts	—
`Sidebar`	Navigation, collapse state, basket badge	route, basket count
`TopBar`	Brand, global search, status pill, demo toggle	health
`SystemStatus`	The always-visible state pill	`/api/health`
`AccountSearch`	Validated input + suggestions + submit	`/api/accounts/search`, `/exists`
`InvestigationSummary`	The seven-tile result strip	`trace.summary`, `trace.performance`
`ProgressTracker`	Real-stage progress	stage events, `meta.processing_ms`
`BlindTestPanel`	Section 36 layout	trace

39.2 Graph

Component	Responsibility
`InvestigationGraph`	Cytoscape host, element lifecycle, selection events
`GraphToolbar`	Zoom, fit, reset, focus, labels, isolate, export
`GraphLegend`	Node/edge encoding + the victim-relative caveat
`NodeInspector`	Section 12
`EdgeInspector`	Section 17
`cytoscapeStyles.js`	The static stylesheet (not a component)

39.3 Timeline

`Timeline` (track + histogram + playhead) · `TimelineControls` (play/pause/speed/bucket) · `TimeRangeSelector` (start/end, presets, clamping) · `TransactionEvent` (one row, selectable, with `source_row_id`).

39.4 Account

`AccountOverview` · `AccountStatistics` · `FlowPanel` (both sides, section 14) · `RiskCard` · `RiskBreakdown` (components + adjustments) · `ClassificationCard` (section 13) · `TerminalEvidenceCard` (section 35 registers) · `TransactionTable` (virtualized, paginated, filterable).

39.5 Evidence · Cases · Documents

`EvidenceBasket` · `EvidenceItem` · `EvidenceSummary` · `PinButton` — `CaseList` · `CaseCard` · `CaseOverview` · `CaseReadiness` — `DocumentPanel` · `DocumentStatus` · `DocumentPreview` · `FreezeAccountPicker`.

39.6 Common

`LoadingState` (never a bare spinner on the critical path) · `ErrorState` (code → copy → one action) · `EmptyState` (message + action) · `Modal` (focus-trapped) · `Toast` · `Button` · `Badge` · `Tooltip` (keyboard-reachable) · `Pagination` · **`ProvenanceChip`** (value + `source_row_id` + verify) · `StatTile` · `MaskedAccount` · `CopyButton`.

---

40. Frontend–Backend Data Contract

Every component that touches the network, and exactly what it uses.

Component	API	Method	Input	Output used
`SystemStatus`, `StatusBar`	`/api/health`	GET	—	`status`, `accounts`, `transactions`, `graph_loaded`, `llm_available`, `ground_truth_available`, `dataset_fingerprint`
`DatasetStatusCard`	`/api/ingest/profile`	GET	—	`row_count`, `unique_accounts`, `date_range`, `unique_transaction_ids`, `duplicate_transaction_ids`, `graph_shape`, `verification`
`IngestPanel`	`/api/ingest` · `/api/ingest/status/{job_id}`	POST · GET	file/path · job_id	`job_id`, `status`, `stage`, `progress`, `warnings`
`AccountSearch`	`/api/accounts/search`	GET	`q`, `limit`, `sort`	`results[]`, `suggestions`
`AccountSearch` (validation)	`/api/accounts/{no}/exists`	GET	account	`exists`
`useInvestigation`	`/api/cases`	POST	victim, incident_time	`case_id`, `next_step`
`InvestigationGraph`	`/api/trace`	POST	victim, case_id, window, limits	`nodes[]`, `edges[]`, `summary`, `performance`, `parameters_used`
`InvestigationGraph`	`/api/graph/{account}`	GET	radius, limits, case_id	`nodes[]`, `edges[]`, `stats`
`GraphToolbar` (expand)	`/api/graph/{account}/expand`	GET	node, max_nodes	new `nodes[]`, `edges[]`
`InvestigationSummary`	(trace response)	—	—	`summary.l1/l2/l3_count`, `total_traced_value`, `performance.total_ms`, `truncated`
`NodeInspector`	`/api/accounts/{account}`	GET	account	`account`, `statistics`, `flow`, `risk`, `classification_hint`, `neighbors`
`ClassificationCard`	`/api/cases/{case}/accounts/{account}/classification`	GET	case, account	`layer`, `hop`, `role`, `confidence`, `confidence_breakdown`, `inputs_used`, `reasons`, `alternative_considered`
`RiskCard`, `RiskBreakdown`	`/api/risk/{account}`	GET	account	`mri`, `band`, `band_edges`, `percentile`, `components`, `adjustments`, `reasons`, `disclaimer`, `calibration_regime`
Graph colouring	`/api/risk/batch`	POST	`account_nos[]`	`mri`, `band`, `layer_hint`, `role`
Dashboard histogram	`/api/risk/distribution`	GET	—	buckets, band counts, top-N
`TerminalEvidenceCard`	`/api/accounts/{account}/terminal-evidence`	GET	account	`terminal_score`, `signals`, `cooccurrence`, `evidence[]`, `interpretation`

Component	API	Method	Input	Output used
`TransactionTable`	`/api/accounts/{account}/transactions`	GET	direction, time, amount, mode, page	`transactions[]`, `pagination`, `totals`
`EdgeInspector`	`/api/transactions/{id}`	GET	transaction_id	`matches[]` (**list**), `is_duplicated`, `citation_hint`
`ProvenanceChip` (verify)	`/api/rows/{source_row_id}`	GET	row id	`raw`, `normalized`, `signals`, `used_in`
`Timeline`	`/api/timeline`	GET	case/account/trace, window, bucket	`events[]`, `buckets[]`, `cumulative[]`, `stats`
`EvidenceBasket` (add)	`/api/evidence/basket`	POST	case_id, account_ids, source_row_ids, edge_ids	`items_added`, `items_skipped`, `skipped[]`
`EvidenceBasket` (read)	`/api/evidence/basket/{case_id}`	GET	case_id	`items[]`, `totals`, `verification`
`EvidenceItem` (remove)	`/api/evidence/basket/{case_id}/{item_id}`	DELETE	ids	`removed`, `item_count`
`CaseList`	`/api/cases`	GET	—	`cases[]`
`CaseOverview`	`/api/cases/{case_id}`	GET	case_id	`case`, `trace`, `layers`, `risk_scores`, `evidence_basket`, `timeline`, `current_holdings`, `documents`, `readiness`
`CaseOverview` (edit)	`/api/cases/{case_id}`	PATCH	fir, officer, notes	updated case + `readiness`
`FreezeAccountPicker`	`/api/cases/{case_id}/holdings`	GET	case_id	`accounts[]`, `freeze_recommended`, `disclaimer`
`DocumentPanel` (diary)	`/api/cases/{case}/documents/case-diary`	POST	format, sections, use_llm	`document_id`, `page_count`, `generation_ms`, `validation`
`DocumentPanel` (freeze)	`/api/cases/{case}/documents/freeze-requisition`	POST	format, selected_accounts, legal_basis	`document_id`, `per_account[]`, `validation`, `warnings`
`DocumentStatus`	`/api/documents/{document_id}/meta`	GET	document_id	kind, size, pages, validation
Download link	`/api/documents/{document_id}`	GET (href)	document_id	the file
`System` benchmarks	`/api/benchmarks`	GET	—	measured values (**or** `null`) + `targets`
`System` evaluation	`/api/eval/summary`	GET	—	`status` / `regime` / metrics or `null`

Not consumed by the frontend in v1.0: `/api/graph/rings` (P2 visual), `/api/eval/confusion-matrix`, `/api/eval/layers`, `/api/eval/threshold-sweep`, `/api/eval/self-validation`, `/api/benchmarks/run` (P2 admin), `/api/ingest/manifest` (shown only on `/system`).

---

41. Loading / Latency Display

The product claims sub-two-second investigation. The UI must **show** that, with measured numbers.

```
TRACE   0.84 s        GRAPH   0.21 s        EVIDENCE   0.08 s
source: backend meta.processing_ms  ·  not a client-side stopwatch
```

Readout	Source
Trace	`trace.performance.total_ms` (and `traversal_ms` / `enrichment_ms` on hover)
Graph	`graph.__meta.processing_ms`
Evidence	`basket.__meta.processing_ms`
Account detail	`account.__meta.processing_ms`
Document	`document.generation_ms`
Ingestion	`manifest.stage_timings_ms`

Rules

1. **Backend timing metadata is preferred wherever it exists.** A client stopwatch includes network and render time and would overstate the backend's cost; it is used only as a labelled fallback ("end-to-end"). **MUST**
2. Latency is displayed **always**, not only when fast. Hiding a slow number is the behaviour of a product with something to hide.
3. Values over the budget (trace > 2,000 ms) render in the warning colour with the budget in the tooltip. Honest and self-aware beats silent.
4. The investigation header keeps the last run's timings visible for the whole session — a judge can glance at them at any point in the demo.
5. These are **measurements**, never targets dressed as results; the `/system` benchmark table keeps the two in separate columns.

---

42. Final Acceptance Tests

The frontend is complete when every box is ticked **on the demo laptop, with Wi-Fi off, against the real 2,000,000-row dataset**.

#	Test	Pass criterion
1	Dashboard loads	Shell + health within 1.5 s
2	Backend health visible	Status pill reflects the true backend state, including when it is down
3	Dataset profile visible	All values from `/api/ingest/profile`; none hard-coded
4	Victim search works	Validation, suggestions, exists-check
5	Investigation starts	`autostart` path runs end to end
6	4-hop trace appears	Nodes to hop 4 where they exist
7	L1 / L2 / L3 appear	Counts match `trace.summary`, lists are clickable
8	Graph renders	Bounded, layered top-to-bottom, legible labels
9	Node selection works	Inspector populates; no layout re-run
10	Edge selection works	Edge inspector with `source_row_id` visible
11	Risk score appears	Score, band, six components, adjustments
12	Risk reasons appear	Rendered **verbatim** from the API
13	Timeline works	Minute filtering, play/pause, scrub, event sync
14	Transactions inspectable	Paginated table + detail view
15	Evidence can be pinned	Accounts, rows and edges; skipped items explained
16	Cases can be created	`POST /api/cases`, id displayed
17	Case can be reopened	`/cases/{id}` rehydrates **without re-tracing**
18	Case diary generates	Document id, page count, validation summary
19	Freeze requisition generates	Account picker, blockers enforced, hub guard respected
20	Documents download	Opens by `document_id`; no path is ever constructed
21	Errors handled	Every code in section 30 has copy and one action; no stack trace reaches the user
22	**No raw 2 M-row data enters the browser**	DevTools network: no response above a few MB; no unbounded array in state
23	**No frontend detection logic**	`grep` finds no MRI arithmetic, no layer assignment, no threshold comparison in `src/`
24	Graph usable at 500+ nodes / 1,500+ edges	Pan/zoom ≥ 30 fps; selection < 100 ms
25	Blind investigation completes quickly	One keystroke to result; total ≤ 3 s
26	**No fabricated data in the UI**	Every number traces to an API field; unmeasured values render `–`

42.1 Three tests worth automating

```
A  "no detection logic" lint
   grep -rE "mri|risk_score|layer" src/ | grep -vE "\.(mri|risk_score|layer)\b"
   any arithmetic hit is a failure

B  "no direct network" lint
   no-restricted-imports: axios outside src/services

C  "renders the backend's reasons" test
   mock /api/risk with a sentinel reason string and assert it appears verbatim
```

43. Implementation Priority

P0 — must work first (the 40% path)

#	Item	Depends on	Done when
1	App shell	—	Renders before health resolves; status pill truthful
2	Backend connection	`services/api.js`	Envelope unwrapped, errors typed, request id surfaced
3	Victim search	`/accounts/search`, `/exists`	Validation + suggestions, Enter submits
4	Investigation page	shell	Five regions, seven states
5	`POST /api/trace`	2, 4	Real stage progress, honest truncation
6	Graph visualization	5	Bounded, layered, legible at 500 nodes
7	Node inspector	6	Opens from trace data, enriches progressively
8	Risk display	7	Components, adjustments, reasons, disclaimer
9	L1 / L2 / L3 display	5	Summary strip + layer lists + case-relative classification

P0 exit: a stranger types an account number, presses Enter, and in under three seconds sees the money trail, the layers, the risk and the reasons.

P1 — becomes an investigation tool

#	Item
10	Timeline with playback
11	Transaction details + `source_row_id` provenance
12	Evidence basket
13	Case management (`/cases` , `/cases/:id`)

P2 — becomes credible and polished

#	Item
14	Case diary generation
15	Freeze requisition with account picker
16	System / benchmark dashboard
17	Advanced graph controls (expand, isolate, rings, export)
18	UI polish: shortcuts, reduced motion, light theme, screen-reader table

Build in this order even if it feels unbalanced. A beautiful case-diary screen on top of a trace that does not render is worth zero; a plain graph that answers the blind test in two seconds is worth 40%.

44. Final Design Principle

> **The frontend is not the brain.**
>
> **It is the investigator's window into the backend.**

```
BACKEND             "Here is what the data says."
                    every number traceable to a row,
                    every reason generated from evidence

FRONTEND            "Here is the clearest way to understand it."
                    layout, emphasis, interaction, honesty about gaps
```

The frontend must never silently alter backend evidence. Not to round a number, not to smooth a label, not to hide an inconvenient state, not to make a chart prettier. If a value is wrong, it is fixed in the backend and the fix flows forward — because the number on the screen and the number in the freeze requisition must be the same number, and a judge is entitled to check.

Three questions for any pull request:

```
1. Could this component produce a number if the backend were deleted?     -> reject
2. Does this render a backend string, or does it compose its own?         -> render it
3. If a judge clicked this value, could we show them the source row?      -> if no, fix it
```

Document control

Version	Date	Author	Change
1.0	2026-10-02	Technical Lead (M1) · M2 Frontend	Initial frontend architecture and UI specification. Eight routes, component inventory, page-by-page layouts with state tables, Cytoscape configuration and performance rules, the complete component→API contract, error and empty-state copy for every backend code, accessibility and visual-language tokens, the observed/inferred/assessed discipline, blind-test and demo modes, acceptance tests and build order. Aligned to Backend API Contract v1.0 and PRD v2.1.

अभेद्य चक्र — the wheel that cannot be broken. The backend proves it; the frontend makes it impossible to misread.